

Bloom Shop (Online Plant Delivery System)

Anisha G.C (240015)

B.Sc. (Hons.) Computing, Softwarica College of IT and E-commerce, Coventry University

ST5009CEM Software Development

Rohit Shrestha

1st August, 2025



Acknowledgement

Over the course of this semester, I had the opportunity to gain practical and theoretical knowledge in designing and developing a fully functional web application. Through this project, I deepened my understanding of key frontend technologies such as HTML, CSS, JavaScript, and React.js, as well as backend tools like Node.js, Express.js, Sequelize, and PostgreSQL. The module also introduced me to essential development practices including version control with Git, API creation and testing using Postman, JWT-based authentication, unit testing with Jest, and effective database modeling.

I improved my problem-solving skills and gained a deeper understanding of the design, development, testing, and deployment of modern web applications by putting these concepts into practice. It was a truly rewarding experience that strengthened my technical background and prepared me for work in more complex development in the future.

I would like to express my gratitude to Mr. Rohit Shrestha for his constant guidance, support, and encouragement throughout this project. I was greatly assisted in completing this project by his insightful lectures and explanations, which made complex subjects understandable. I am also grateful for the constructive criticism and support I got throughout the semester, as this greatly impacted my learning and growth.

Abstract

During this semester in the ST5007CEM Web Development module, I worked on building a plant shopping website that reflects both my technical skills and practical understanding of modern web development. The project gave me a chance to apply classroom knowledge to a real-world scenario and face the challenges of creating a fully functional online application.

The development process involved building user interfaces using React.js, handling backend logic with Node.js, and managing a relational database using PostgreSQL with Sequelize. Implementing features such as user authentication, product management, and a favorites and cart system helped deepen my understanding of full-stack application flow and secure user interaction. I also gained valuable experience in writing and running test cases using Jest and Supertest to ensure functionality and reliability.

Overall, this project has significantly improved my problem-solving skills, strengthened my grasp of web development tools and frameworks, and boosted my confidence in working independently on real-world applications. It has been a rewarding and formative learning experience.

Table of Contents

Acknowledgement	2
Abstract.....	3
Introduction	10
Aim	11
Objectives	12
Scope.....	13
Design Process	14
Functional Requirement	15
Non-Functional Requirement	16
Prototype	17
Home Page	17
Login Page	18
Signup Page	18
User Dashboard	19
Product Detail Page.....	19
Plant Care Page	20
Cart Page	20
Delivery info Page	21
Order Confirmation Page.....	21
Favorite page	22
Admin Page	22
System Architecture.....	23
Use Case Diagram	24
ER Diagram.....	25
Tools.....	26
Technology.....	27
Frontend Technologies	28
Backend Technologies.....	29
Database Technologies	30
Implementation	31
Backend Folder Structure	31
Database	32

Db.js	32
Model	33
userSchema.js	33
Productschema.js	34
favoriteModel.js	35
DeliveySchema.js	36
Controller	37
authController.js	37
userController.js	39
productController.js	40
favoriteController.js	42
deliveryController.js	43
fileController.js	44
Route	44
userRoute.js	44
authRoutes.js	45
productRoutes.js	45
favoriteRoutes.js	46
deliveryRoutes.js	46
uploadRoutes	46
Middleware	47
multerConfig.js	47
token-middleware.js	48
Verifytoken.js	49
Security	49
helper.js	49
Jwt-utils.js	50
Index.js	50
Frontend folder Structure	52
Home.jsx	54
Login.jsx	56
Signup.jsx	58
Admin.jsx	59

Delivery.jsx	63
Database	69
Table.....	69
user table	70
Delivery table	70
Product table.....	71
Testing and Evaluation	71
Unit Testing	73
Test Case and Expected Results	74
Future Work	75
Conclusion.....	77
References	78
Appendix	79

Table of Figure

Figure 1: Plant Store Infographics.....	10
Figure 2: Aims	11
Figure 3: Objectives.....	12
Figure 4: Scope.....	13
Figure 5: Design Process	14
Figure 6: Home Page	17
Figure 7: Login Page	18
Figure 8: Signup Page.....	18
Figure 9: Home Page	19
Figure 10: Product Detail Page.....	19
Figure 11: Plant Care Page	20
Figure 12: Cart Page	20
Figure 13: Delivery Page	21
Figure 14: Order Confirmation.....	21
Figure 15: Favorite Page	22
Figure 16: Admin Page	22
Figure 17: System Architecture.....	23
Figure 18: Use Case Diagram	24
Figure 19: ER Diagram.....	25
Figure 20: Tools Used.....	26
Figure 21: Technology Stack	27
Figure 22: Frontend Technology	28
Figure 23: Backend Technology	29
Figure 24: Database Technology.....	30
Figure 25: Backend Folder Structure	31
Figure 26: Db.js code.....	32
Figure 27: userSchema.js	33
Figure 28: productSchema.js	34
Figure 29: favoriteModel.js.....	35
Figure 30:deliverySchema.js	36
Figure 31: authController.....	37

Figure 32:userController.js	39
Figure 33: productController.js.....	40
Figure 34: favoriteController.js	42
Figure 35: deliveryController.js.....	43
Figure 36: fileController.js.....	44
Figure 37: userRoute.js	44
Figure 38:authRoute.js.....	45
Figure 39:productRoute.js	45
Figure 40: favoriteRoute.js.....	46
Figure 41: deliveryRoute.js	46
Figure 42: uploadRoutes.js	46
Figure 43:multerConfig.js.....	47
Figure 44: token-middleware.js	48
Figure 45:verifyToken.js.....	49
Figure 46:helper.js	49
Figure 47: jwt-utils.js.....	50
Figure 48:index.js	50
Figure 49:Home.jsx	54
Figure 50: Home frontend	56
Figure 51:login.jsx	56
Figure 52: Login frontend.....	58
Figure 53: Signup.jsx	58
Figure 54: Signup frontend	59
Figure 55: Admin.jsx.....	59
Figure 56: Admin frontend.....	63
Figure 57: Delivery Frontend	67
Figure 58:User Dashboard frontend	67
Figure 59: Favorite Frontend	68
Figure 60: Cart Frontend	68
Figure 61: Order Confirmation frontend	68
Figure 62: Plant Care Fronend	69
Figure 63: user table	70

Figure 64: delivery table	70
Figure 65: product table.....	71
Figure 66: Testing and Evaluation.....	72
Figure 67: Future Work illustration.....	76

Introduction

In today's digital world, online shopping has become the go-to way for people to buy almost everything from clothes to electronics. But one area that still feels a bit left out is plant shopping. Plants are more than just decorations they help clean the air, make our homes feel more peaceful, and even improve our mood. Still, many people find it hard to buy healthy, good-quality plants nearby. Most of the time, it means going from one nursery to another without much help or information about how to care for the plants, which can be frustrating and lead to poor buying choices. [*\(The Ultimate Guide to Selling Plants Online, 2025\)*](#)

That's where Bloom Shop comes in. It's an online platform designed to make buying plants easier and more convenient. With Bloom Shop, users can browse through a variety of indoor and outdoor plants, read care tips, and order them directly from their devices no need to leave the house. The platform is built using modern web technologies and is designed to work smoothly on phones, tablets, and computers. It includes useful features like secure login, a shopping cart, easy checkout, a search bar, and an admin dashboard for managing everything behind the scenes. Bloom Shop isn't just about selling plants it's about connecting plant lovers, local nurseries, and plant sellers all in one place.



Figure 1: Plant Store Infographics

Problem Statement

Buying plants the usual way can be tiring and limited. You often have to visit multiple nurseries just to find the right one, and it's hard to tell if a plant is healthy or how to take care of it properly. Many local nurseries don't even sell online, so they miss out on reaching more customers. There also aren't many platforms that give aftercare help, which leads to unhappy buyers and dead plants. This project aims to solve that by creating an online plant shopping system where sellers can easily list plants and buyers can shop, learn, and order with ease. ([Lakhani, 2025](#))

Aim

To develop a user-friendly web-based platform that enables customers to browse, search, and purchase a variety of indoor and outdoor plants online, while providing personalized plant care information, secure user authentication, and streamlined order management to enhance the overall plant shopping experience.

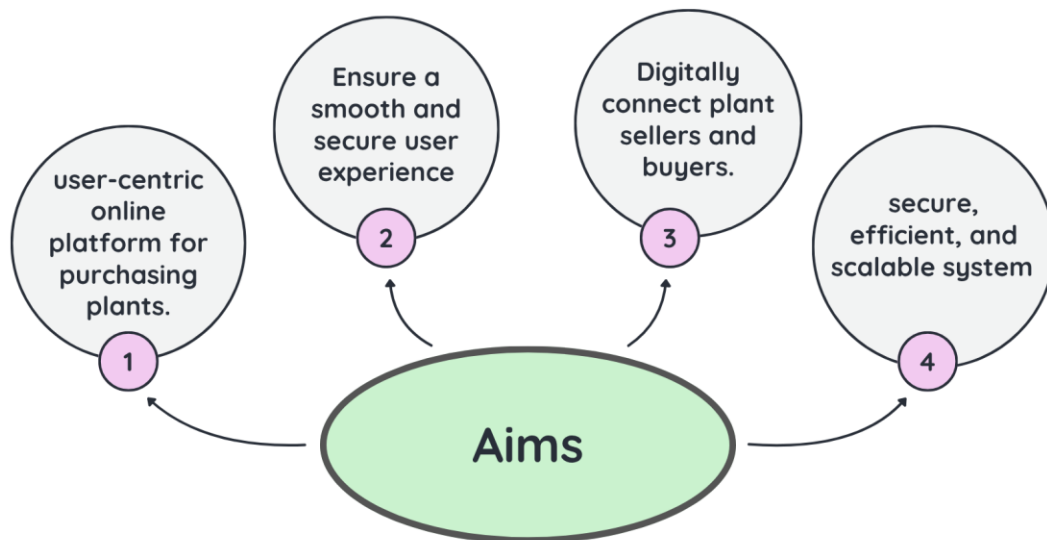


Figure 2: Aims

Objectives

1. To design and develop a responsive plant shopping website
2. To implement secure user authentication using JWT
3. To provide a detailed product catalog with plant info
4. To enable cart management and seamless order placement
5. To build an admin interface for plant and order management
6. To provide personalized plant care tips post-purchase

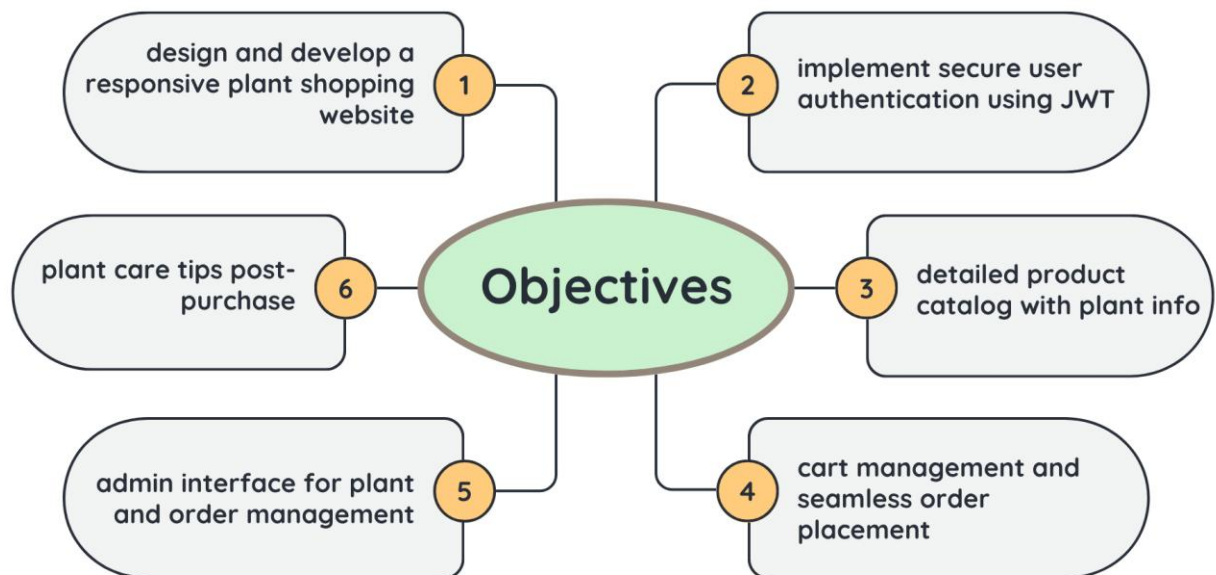


Figure 3: Objectives

Scope

This project focuses on building the main features needed for an online plant shopping system. Users and admins can register and log in securely. Admins can manage products—adding, editing, or removing plant listings with images and details. Users can browse plants, add them to their cart, and place orders through a simple checkout process. There’s also real-time order tracking and a dashboard for admins to manage users, orders, and plant listings efficiently.

Out of Scope: To keep things simple, this version doesn’t include third-party payment gateways, delivery tracking, in-app messaging, or AI-based plant recommendations. These could be added in future updates.

Technology Stack: The platform uses modern tools for full-stack development. The frontend is built with React.js for a responsive UI. The backend uses Node.js and Express, with PostgreSQL as the database and Sequelize to manage data. JWT handles secure authentication for both users and admins.

Intended Users: There are two main users: buyers and admins. Buyers can browse and buy plants easily, while admins handle listing updates, orders, and overall platform management.

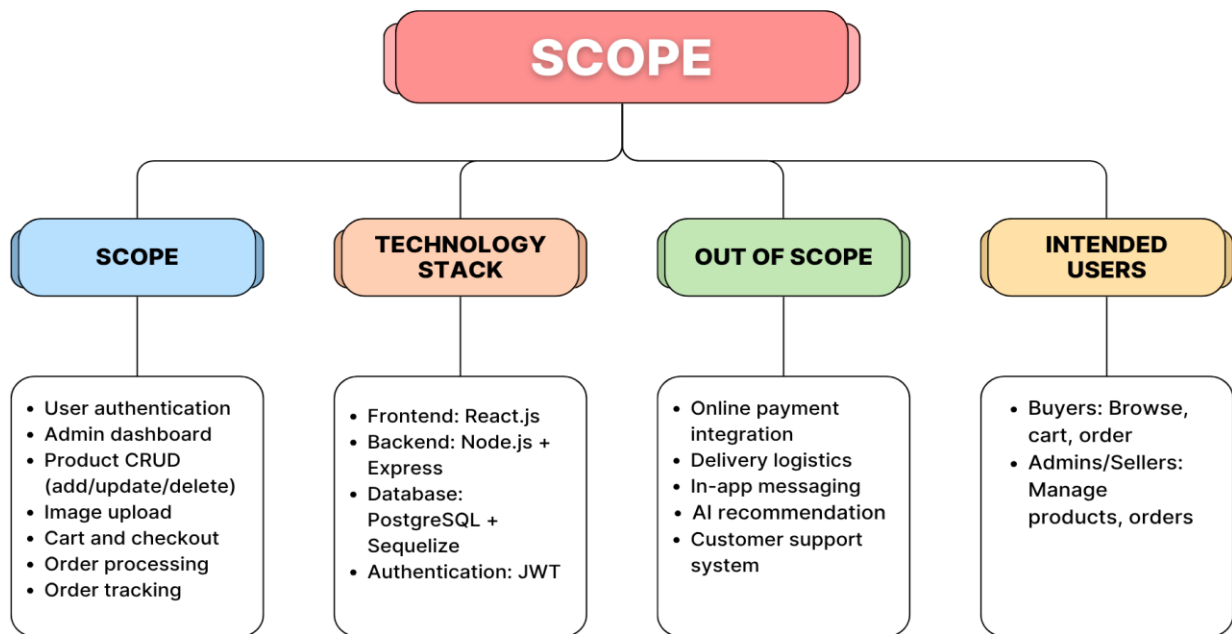


Figure 4: Scope

Design Process

Requirement Gathering: The main goal is to identify providing an intuitive platform for browsing, purchasing, and managing plant products, along with user authentication and a favorites feature.

Wireframing: It is done using Balsamiq to visualize the layout of pages such as the home page, product detail page, login/signup forms, dashboard, cart, and admin panel.

Database Design: It is planned using PostgreSQL, ensuring relational tables for users, plants, favorites, and orders. The Sequelize was used to model these relationships effectively, supporting secure and scalable data management. ([*Web Design Process Explained in 8 Easy Steps, n.d.*](#))

UI/UX: The design focused on accessibility and visual appeal using React.js, with pastel color themes and clean white backgrounds for a calming shopping experience. Interactive components like buttons, modals, and routing were handled with React Router DOM.

System Architecture: the system was designed with a clear structure, frontend handles the user interface using React and Vite for speed, while the backend runs on Node.js and Express with secure routes. Data is stored in a PostgreSQL database, and Sequelize helps manage everything smoothly.

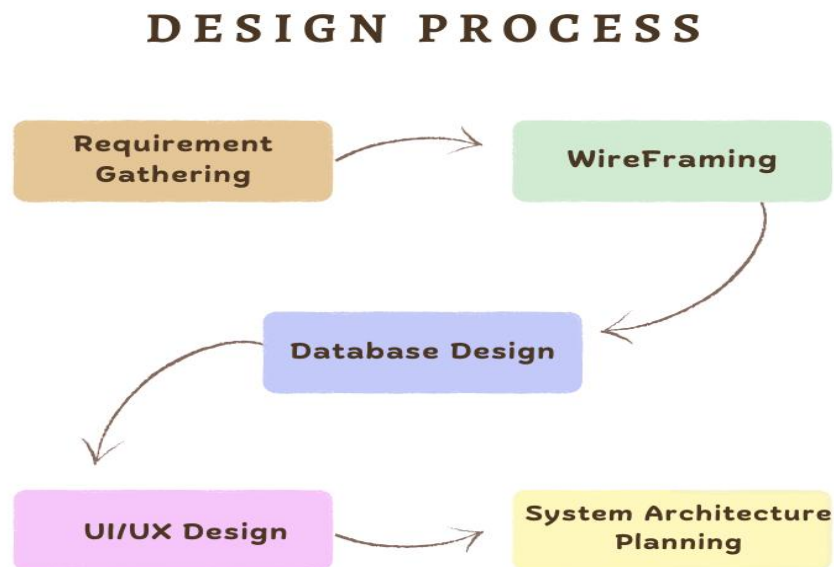


Figure 5: Design Process

Functional Requirement

The system includes essential functional requirements such as register, log in securely with JWT, and browse or purchase plants. Admins can manage products, orders, and inventory. Users can search plants, add to cart, and place orders. ([Agilemania, n.d.](#))

S.N	Functional Requirement	Description
1	User Registration and Login	The system must enable users to Register, Login and Logout securely using email and password
2	Secure Authentication (JWT)	User Session is securely managed with token-based authentication
3	Role-Based Access	Admins have full access to manage plants and orders. Users can shop and view
4	Product Management	Admins can add, edit, or delete plant listings with name, price, image, etc.
5	Search and Filter Product	Users can search plants by name
6	Cart Management	The system should offer various payment methods and handle online transactions securely.
7	Order Placement	Users can place order and view their order
8	Admin Dashboard	Admins can view user activities, total sales, and manage plant inventory.
9	Plant Care Information	Each plant has after care and care tips (watering, light, etc.) displayed on its detail page.
10	Stock Management	Tracks available quantity and updated stock automatically after purchase

Non-Functional Requirement

The system is built to grow easily with more users and orders, while keeping everything secure using JWT and bcrypt. It runs fast with optimized queries and smooth page transitions. The interface is simple to use, works well on all devices, and stays reliably online.

S.N	Non-Functional Requirement	Description
1	Scalability	The system must support growth by handling an increasing number of users, products, and orders without performance degradation.
2	Security	User data and passwords must be protected using encryption techniques like bcrypt hashing and secure JWT-based authentication, along with role-based access control.
3	Performance	The platform should deliver fast response times, optimized database queries, and smooth navigation during peak usage.
4	Usability	The UI must be intuitive, consistent, and responsive across all device sizes to ensure a seamless shopping experience.
5	Availability	The system should be accessible 24/7 with minimal downtime, ensuring consistent access for buyers and sellers.
6	Maintainability	Code should be modular and well-documented (frontend and backend in separate folders), making it easier for future developers to maintain and upgrade.
7	Responsiveness	The system must adapt well to different screen sizes and resolutions, offering a mobile-friendly layout and interactions.
8	Data Integrity	All plant and user-related data must be stored and updated accurately, preventing data loss or corruption during transactions or system failures.

Prototype

A prototype is an early version of a system that visually represents the user interface and core functionalities before the full development process begins. It serves as a preliminary model that allows developers, users, and stakeholders to interact with the system concept, providing a clear idea of how the final product will look and function. By using a prototype, teams can gather valuable feedback, identify usability issues, and refine design elements early on, which helps improve the overall user experience and reduce costly changes later. Whether simple sketches or detailed interactive models, prototypes play a crucial role in guiding the development process, ensuring that the final system meets user needs and expectations efficiently.

Home Page

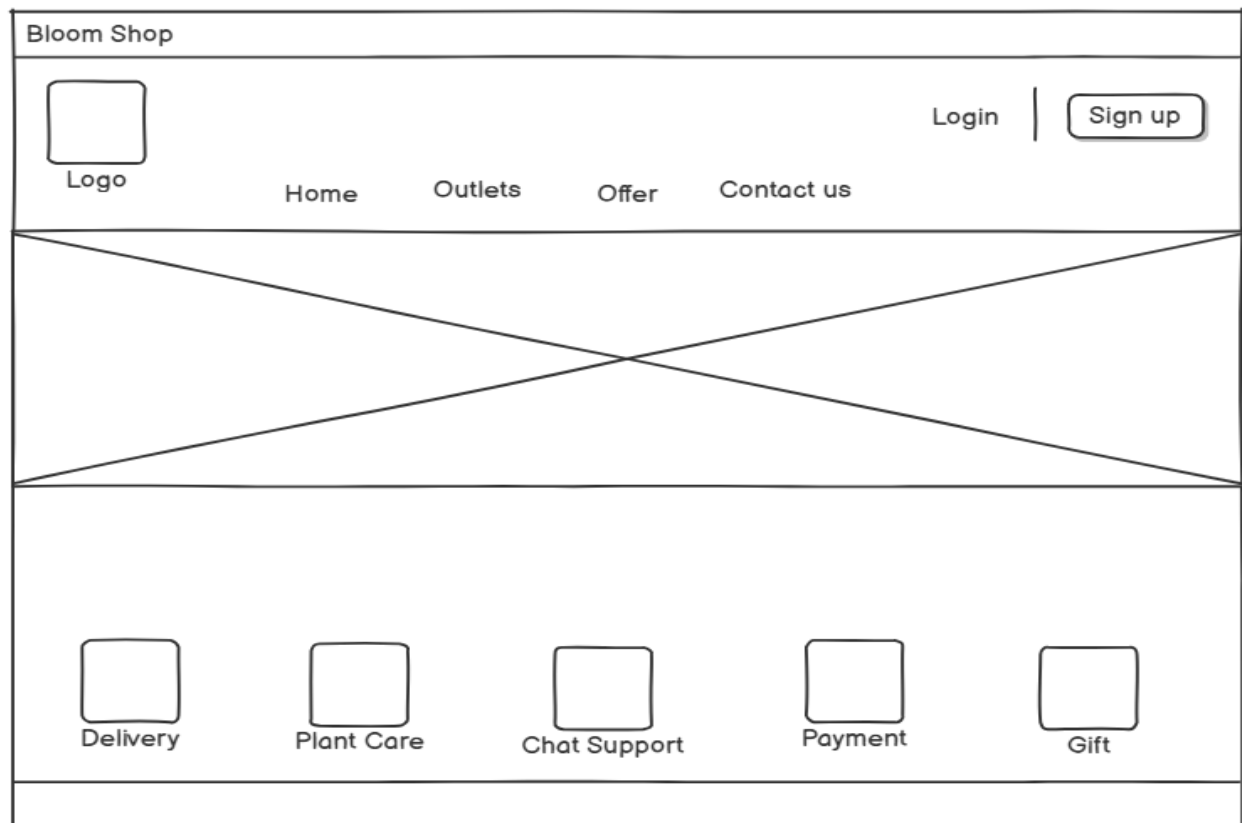
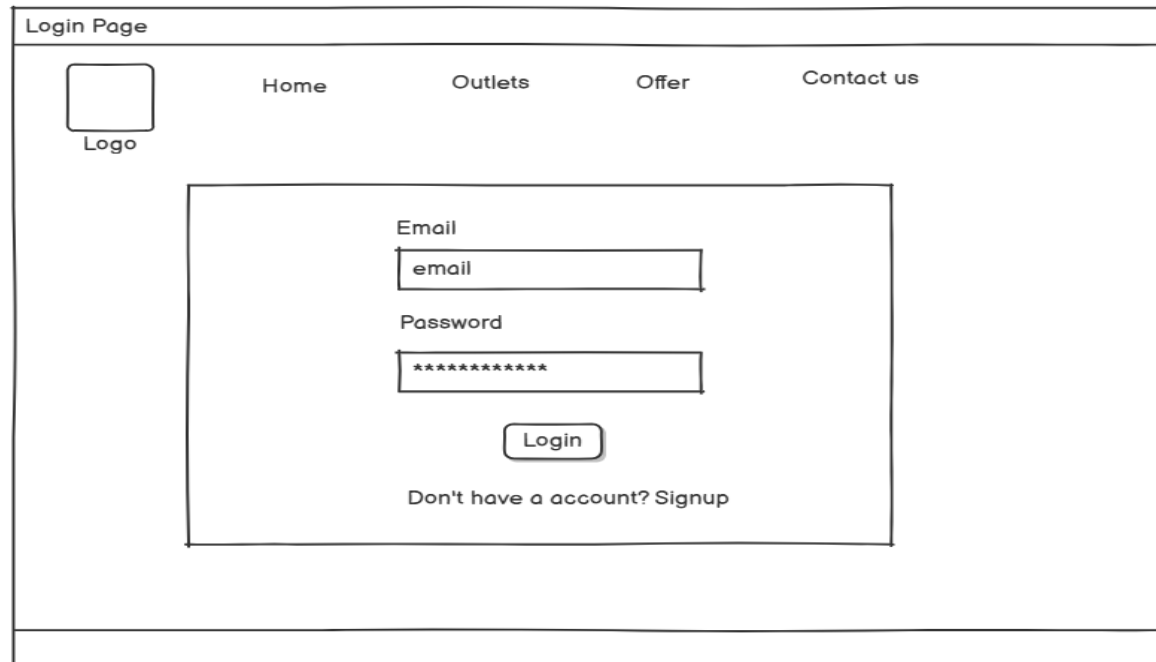


Figure 6: Home Page

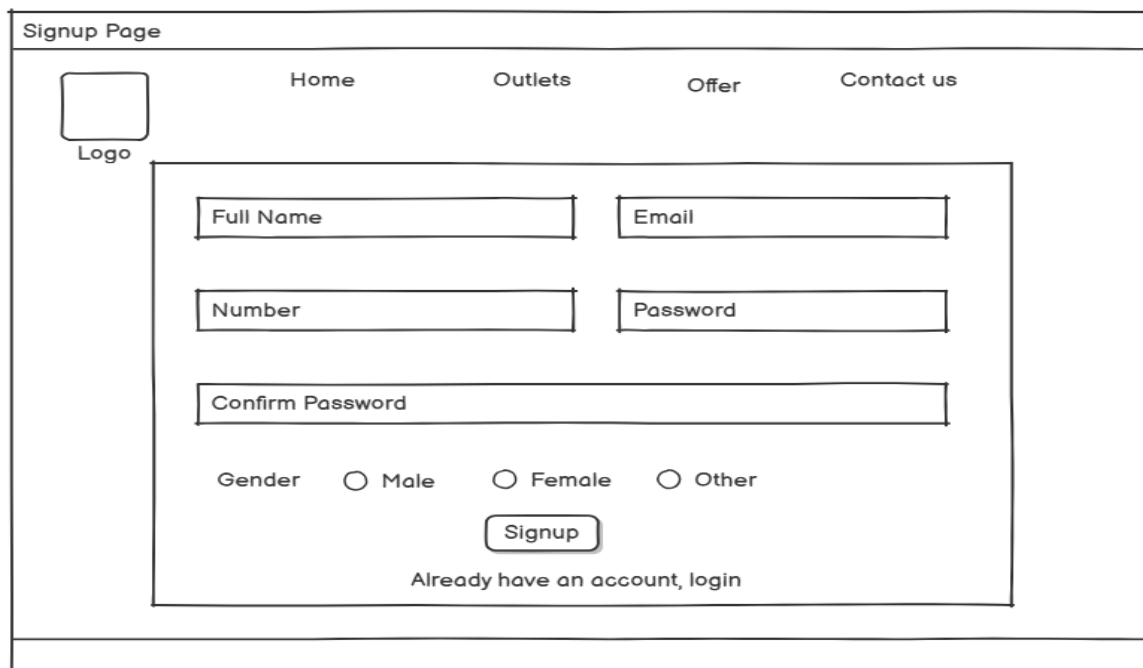
Login Page



The Login Page UI mockup features a header bar with the title "Login Page" on the left and navigation links "Home", "Outlets", "Offer", and "Contact us" on the right. Below the navigation links is a "Logo" placeholder. The main content area is a large rectangle containing a centered form. The form has two input fields: "Email" with the placeholder text "email" and "Password" with masked characters "*****". Below these fields is a "Login" button. At the bottom of the form is a link that says "Don't have a account? Signup".

Figure 7: Login Page

Signup Page



The Signup Page UI mockup features a header bar with the title "Signup Page" on the left and navigation links "Home", "Outlets", "Offer", and "Contact us" on the right. Below the navigation links is a "Logo" placeholder. The main content area is a large rectangle containing a centered form. The form has five input fields: "Full Name", "Email", "Number", "Password", and "Confirm Password". Below these fields are three radio buttons for "Gender" with options "Male", "Female", and "Other". Below the radio buttons is a "Signup" button. At the bottom of the form is a link that says "Already have an account, login".

Figure 8: Signup Page

User Dashboard

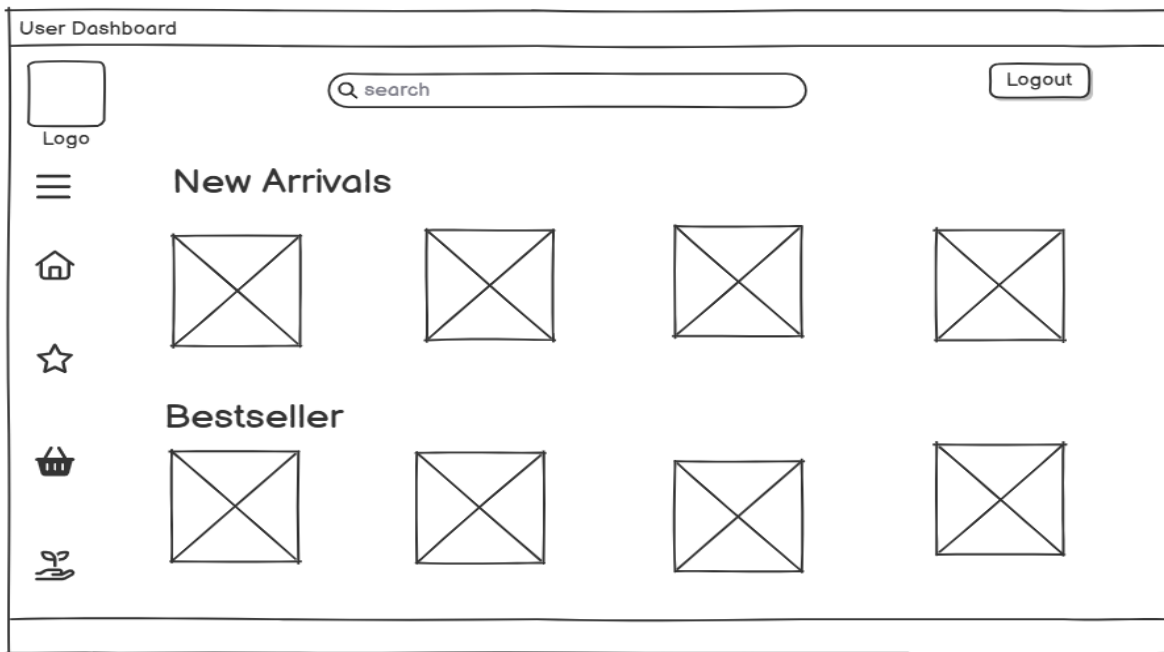


Figure 9: Home Page

Product Detail Page

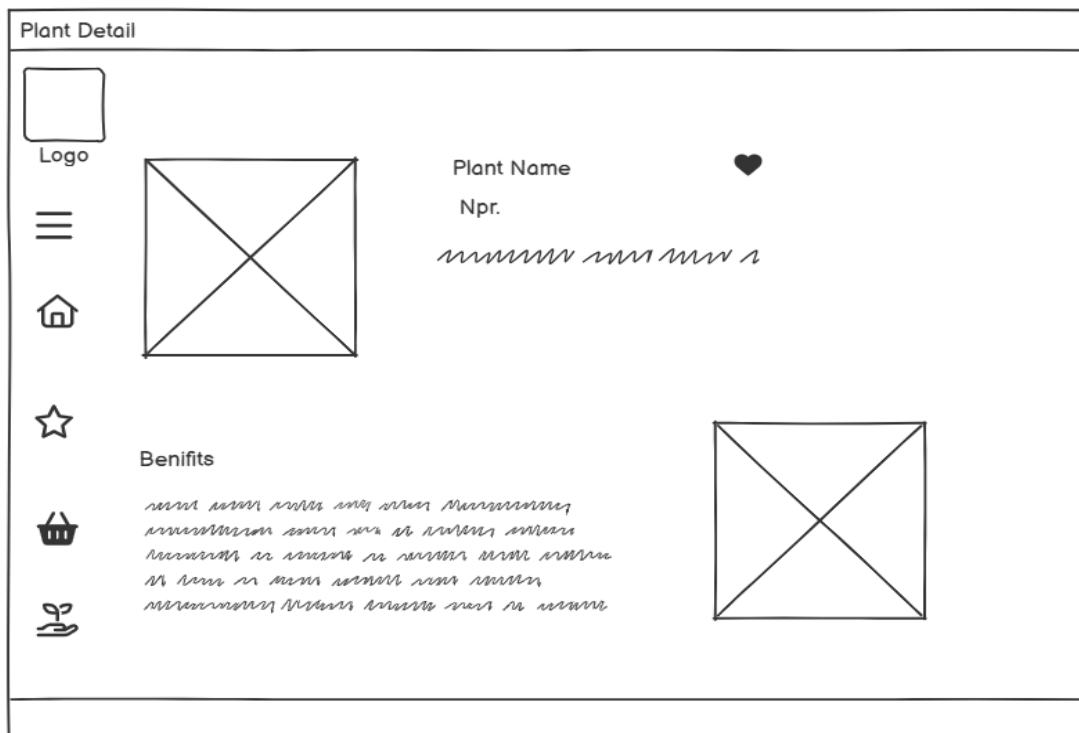


Figure 10: Product Detail Page

Plant Care Page

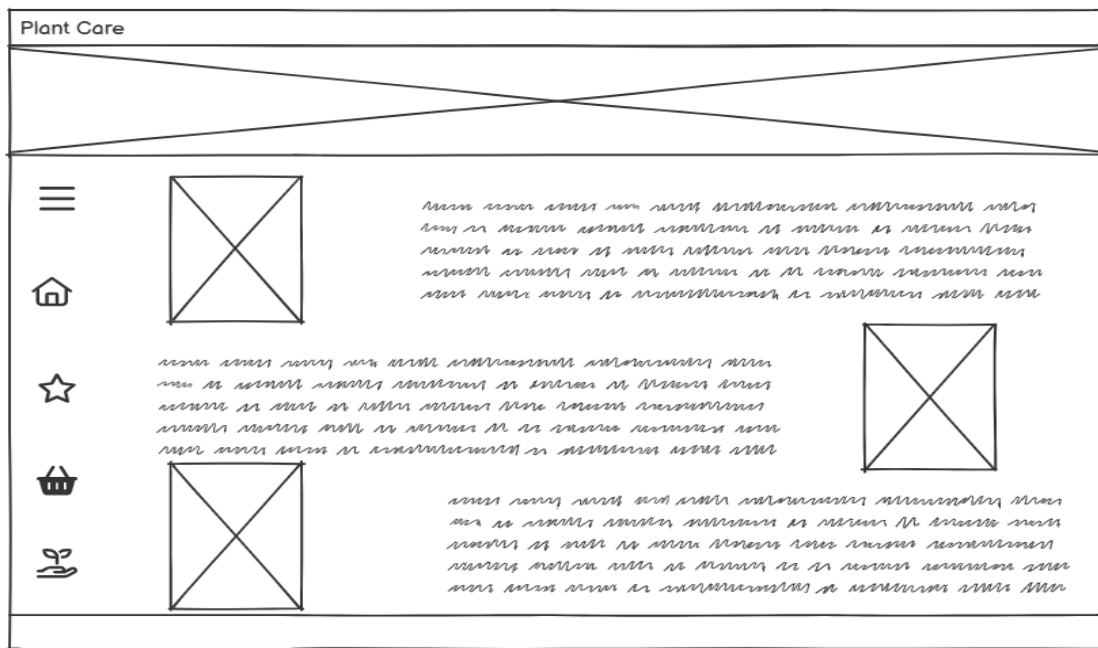


Figure 11: Plant Care Page

Cart Page

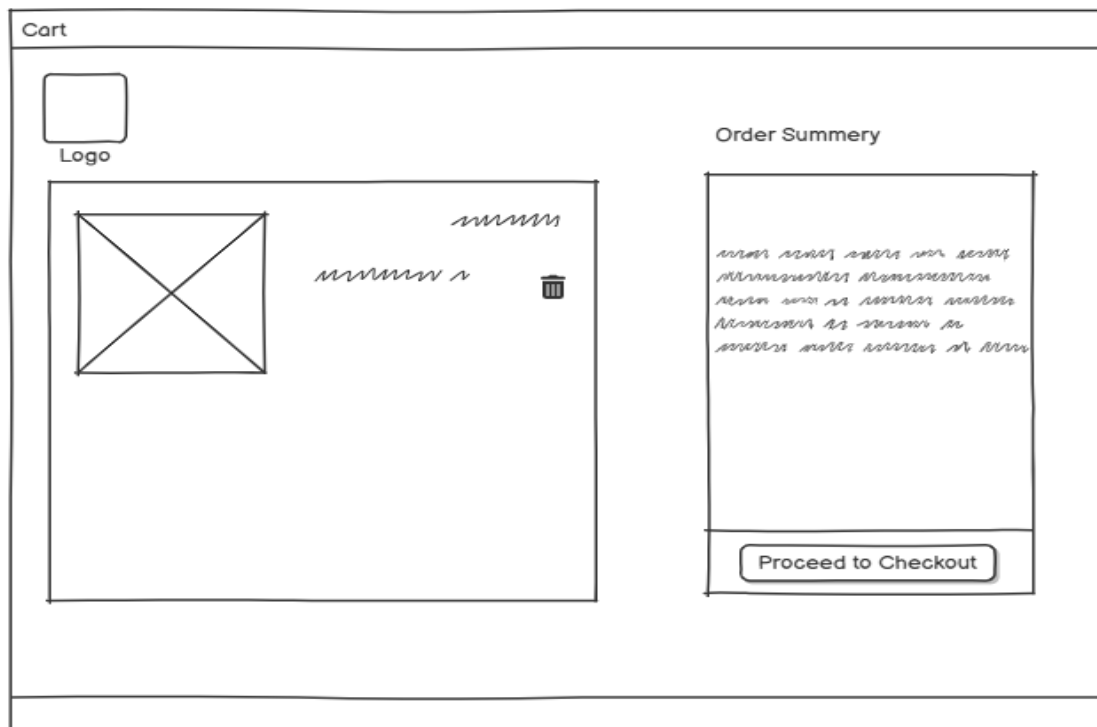


Figure 12: Cart Page

Delivery info Page

Delivery Information	
 First Name Last Name Phone Number Email City Payment Method	Special Instruction(Optional) Order Summary ~~~~~ ~~~~~ ~~~~~ ~~~~~ ~~~~~ ~~~~~ Place an Order

Figure 13: Delivery Page

Order Confirmation Page

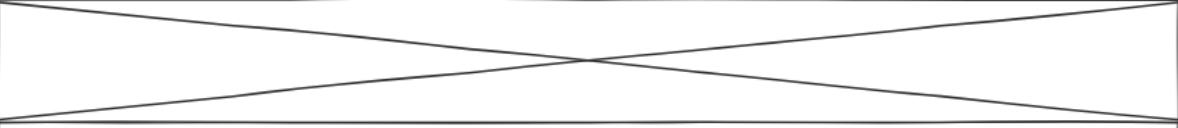
Order Confirmation	
 Logo	 ~~~~~ ~~~~~ ~~~~~ ~~~~~ ~~~~~
	

Figure 14: Order Confirmation

Favorite page

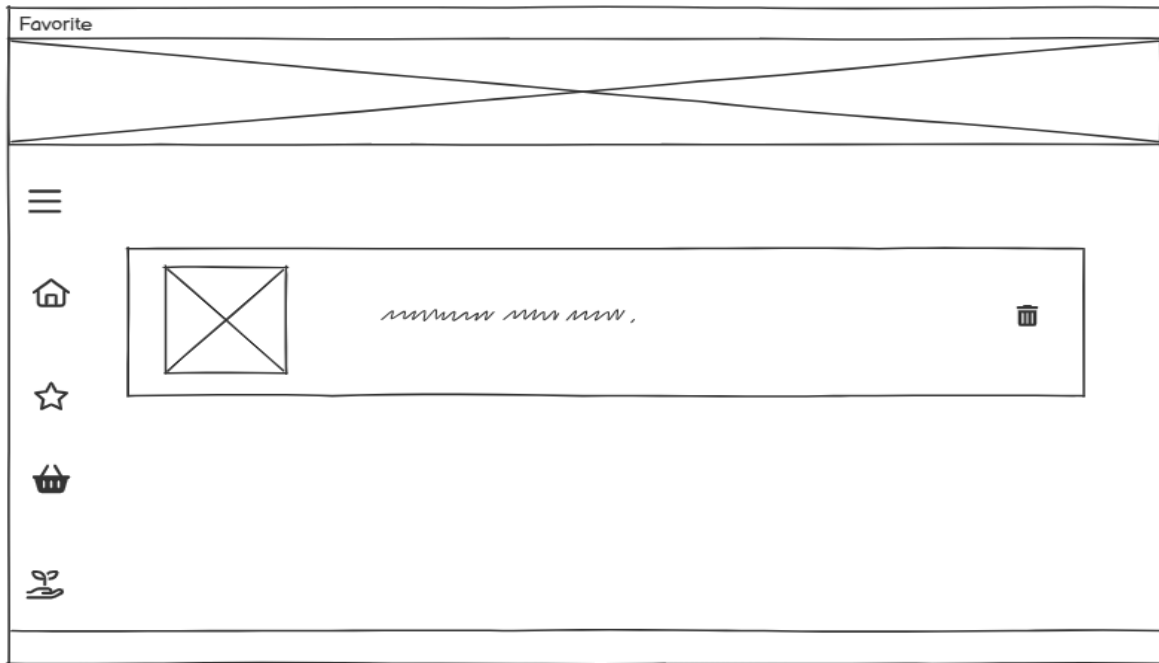


Figure 15: Favorite Page

Admin Page

Admin Page

Logo

Admin Pannel

Product Id

Product Name

Image

Quantity

Price

ADD

#	Product ID	Product Name	Image	Quantity	Price		

Figure 16: Admin Page

System Architecture

Below layered architecture ensures smooth data flow, enhances security, and makes it easy to extend or maintain individual components without affecting the entire system. I followed a three-tier architecture consisting of the frontend, backend, and database layers. This structure helps keep everything organized, secure, and easy to maintain or update without affecting the whole system.

The frontend is built with React.js, which allows for a responsive and interactive user interface. I used React Router DOM for smooth navigation and Axios to connect the frontend with the backend through RESTful APIs. Users can browse plants, view details, manage their cart, and add items to favorites.

The backend is developed using Node.js with Express.js. It handles all the core logic, processes client requests, and manages authentication using JWT tokens to protect user data and secure routes. The backend also supports CRUD operations for users, products, favorites, and orders. [\(Gillis, 2024\)](#)

For the database, I used PostgreSQL to store structured data like user details, plant inventory, favorites, and orders. Sequelize ORM helps manage the models and perform database operations efficiently.

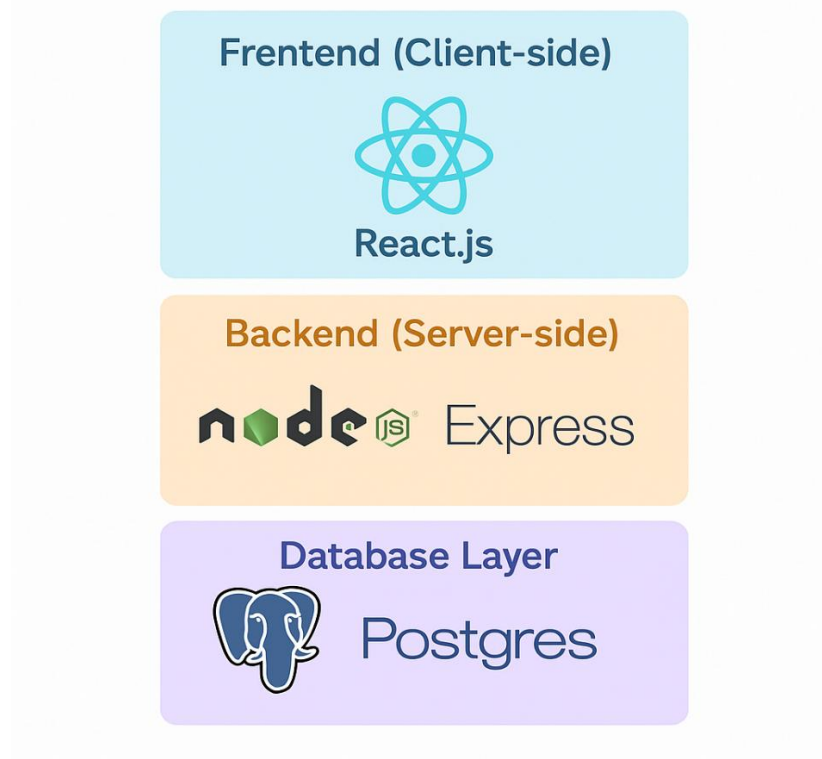


Figure 17: System Architecture

Use Case Diagram

This use case diagram gives a clear overview of how users interact with an e-commerce system. There are two main users: Customers and Admins. Customers can register and log in to access features like browsing products, adding items to their cart or favorites, and checking out all from their personalized dashboard. They also have the ability to log out anytime. The actions like "Add to Cart" and "Add to Favorite" are shown as optional extensions of the user dashboard. On the admin side, after logging in, admins access a central admin panel where they can upload images, add new products, and manage inventory. Inventory management includes updating or deleting existing products, giving admins full control over the product catalog. The diagram uses solid lines to show direct user interactions and dashed lines with labels like <<include>> and <<extend>> to indicate linked or optional features. Overall, it highlights how each role interacts with system functionalities.

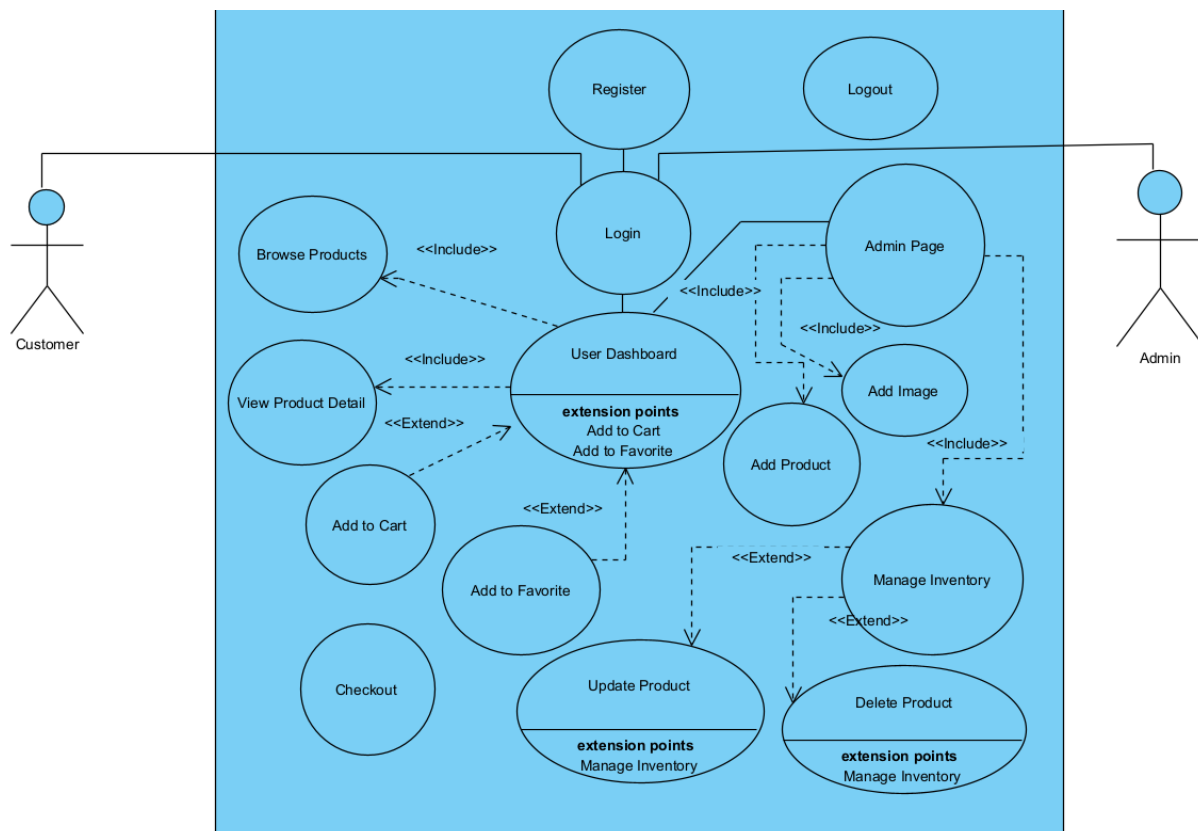


Figure 18: Use Case Diagram

ER Diagram

This ER diagram outlines the key structure of Bloom Shop by defining its main entities and their relationships. At the core is the User entity, identified by `userId`, storing essential information such as name, email, password, and gender. This enables personalized user experiences within the system.

The Product entity includes attributes like `productId`, name, quantity, price, and image. It also contains `userUserId`, indicating a link to the user who listed the product useful in multi-vendor systems.

To handle many-to-many relationships, the system uses junction tables. The Favorite table connects users and products, allowing each user to favorite multiple products, and each product to be favorited by multiple users. The Cart table links users to the products they plan to purchase, supporting multiple items per user.

The Order entity captures transaction details such as `orderId`, `orderDate`, and `totalPrice`. It is connected to the Product_Order table, which links specific products to specific orders, supporting orders with multiple items.

Primary and foreign keys maintain data integrity across all tables. Overall, this ER diagram provides a clear and scalable structure for managing users, products, favorites, shopping carts, and orders in an efficient e-commerce application.

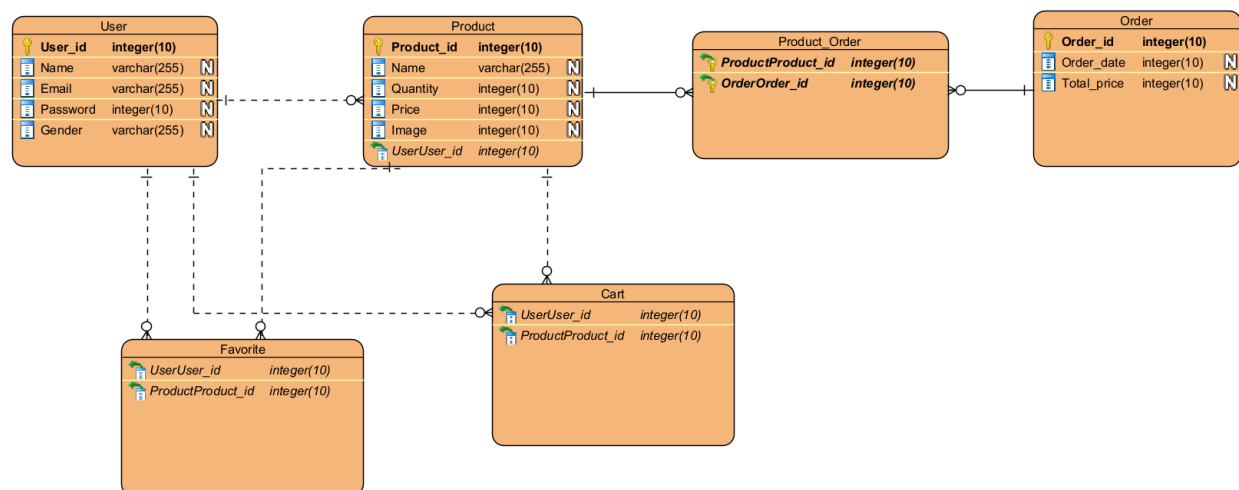


Figure 19: ER Diagram

Tools

Development Tools: Visual Studio Code was used for both frontend and backend work due to its ease of use and helpful extensions. The interface was built using HTML5, CSS3, JavaScript, and React.js, with Vite streamlining the process through fast builds and live reloads.

Backend & Database: On the backend, Node.js and Express.js were used to build APIs, while Sequelize simplified interactions with the PostgreSQL database.

Design & Prototyping: To design the interface, I used Figma for high-fidelity mockups and Balsamiq for low-fidelity wireframes during the early stages. Canva was used for creating graphics and visuals for our reports and presentations.

Testing & Debugging: To ensure everything ran smoothly, Postman was used for testing API endpoints, and Jest helped with writing unit and integration tests for the backend.

Collaboration & Documentation: Git and GitHub supported version control and collaboration, while Microsoft Teams was used for communication with peers. Documentation was neatly compiled in Microsoft Word, and most technical issues were resolved through quick research on Google.

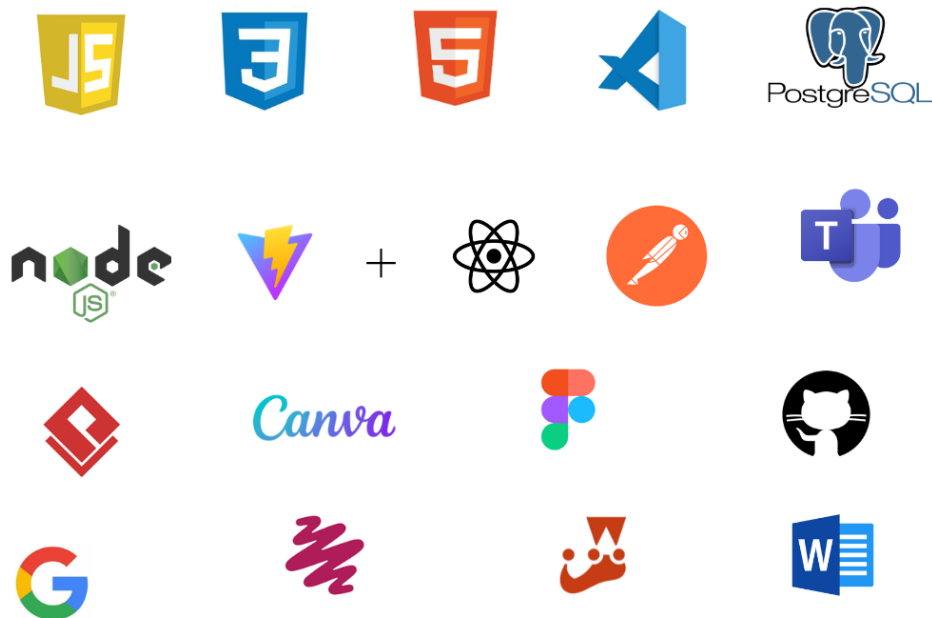


Figure 20: Tools Used

Technology

JavaScript played a vital role in the project as both the frontend and backend were developed using JavaScript-based technologies. React.js, a JavaScript library, was used for building dynamic user interfaces, while Node.js (JavaScript runtime) with Express.js managed backend APIs and server-side logic. JavaScript enabled seamless communication between the frontend and backend using Axios. Its asynchronous nature supported real-time interaction, and libraries like bcrypt.js and JWT ensured secure authentication processes. Overall, JavaScript provided a full-stack solution for fast, scalable, and maintainable development. ([*What Is a Tech Stack: Examples, Components, and Diagrams, n.d.*](#))

Technology Stack

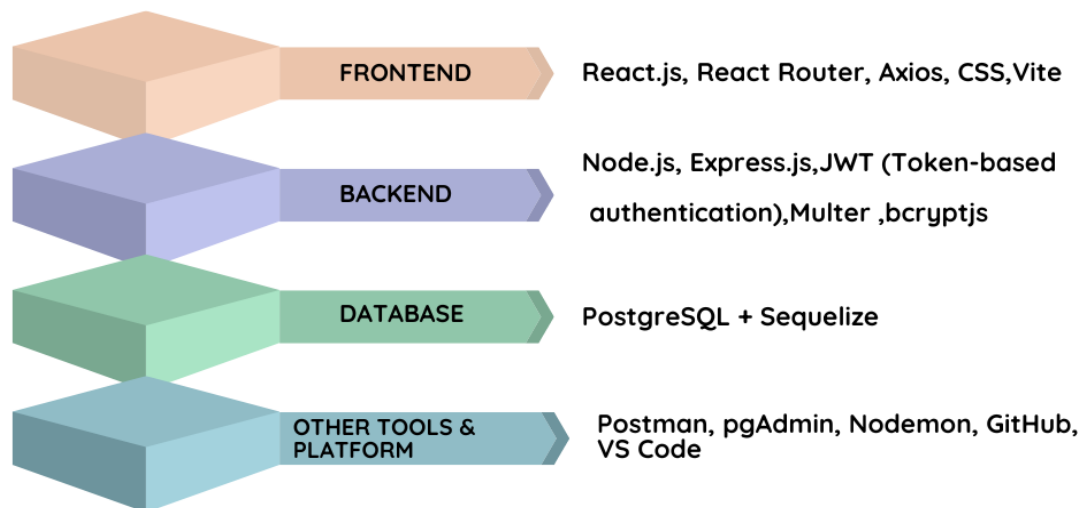


Figure 21: Technology Stack

Frontend Technologies

The frontend of the Online Plant Shopping System was developed using React.js with Vite as the fast build tool, ensuring optimal performance and a smooth development experience. React allowed for building reusable UI components such as the homepage, product detail pages, cart, and admin dashboard.

→ CSS was used to style the interface and ensure responsiveness across devices. React Router DOM provided seamless client-side routing, enabling users to navigate between pages like login, register, and dashboard without reloading the page.

→Axios was used to communicate with the backend API, sending and receiving data such as user info, products, and favorites efficiently.

→react-hook-form simplified form handling for features like registration, login, and admin product uploads offering better validation, cleaner code, and faster performance.

→jwt-decode helped extract user information from JWT tokens stored in the frontend for managing authentication state. ([*What Is the Difference Between Front-End and Back-End Development?, 2024*](#))

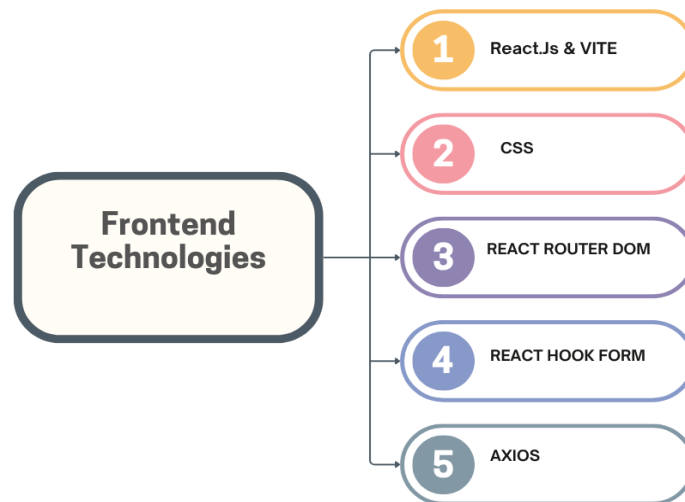


Figure 22: Frontend Technology

Backend Technologies

The backend of the Online Plant Shopping System was built using Node.js with the Express.js framework to manage routes, handle API requests, and implement server-side logic efficiently.

→Sequelize ORM was used for database modeling and querying with PostgreSQL, allowing smooth interaction with the relational database using JavaScript syntax.

→JWT (JSON Web Token) was implemented for secure user authentication and route protection. Tokens were generated on login and verified for accessing protected routes like dashboard, favorites, or adding products.

→bcrypt.js was used to hash user passwords before storing them in the database, ensuring password security.

→Multer handled file uploads (such as plant images) by storing them locally in the uploads/ directory, which was later retrieved to display plant products.

→CORS middleware was configured to allow safe interaction between your frontend (React) and backend (Express) hosted on different ports during development.

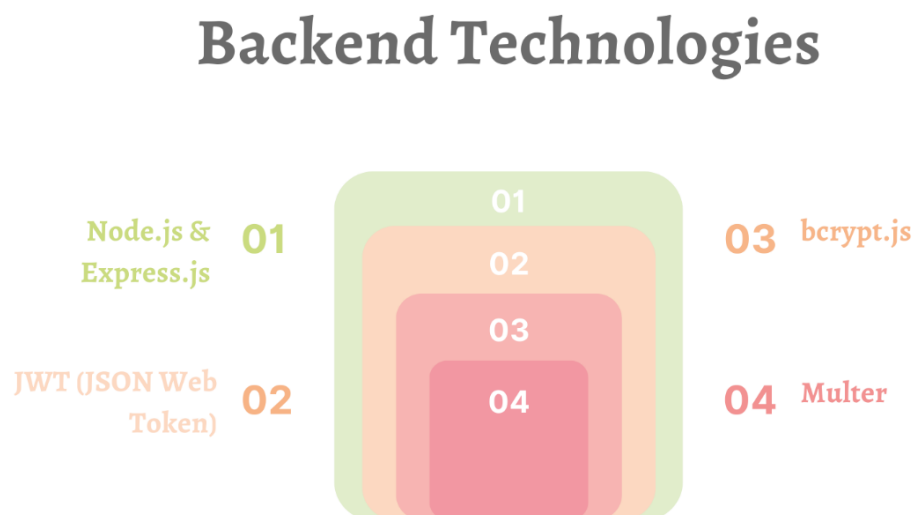


Figure 23: Backend Technology

Database Technologies

It uses PostgreSQL as the primary database technology due to its robustness, scalability, and efficiency in handling structured relational data. It stores all critical information including user details (such as name, email, password, and gender), plant product information (name, price, quantity, image, and description), user favorites, and delivery-related data like address, phone, city, and payment method. To simplify interaction with the database, the project utilizes Sequelize ORM, which allows defining tables as JavaScript models and executing queries using object-oriented syntax. This approach improves productivity, manages relationships between tables effectively, and handles migrations smoothly. Additionally, PgAdmin4 was used as a graphical interface to manage and monitor the database visually, helping during testing and debugging. This combination ensures secure, reliable, and efficient data handling throughout the system.

Database Technology



Figure 24: Database Technology

Implementation

Backend Folder Structure

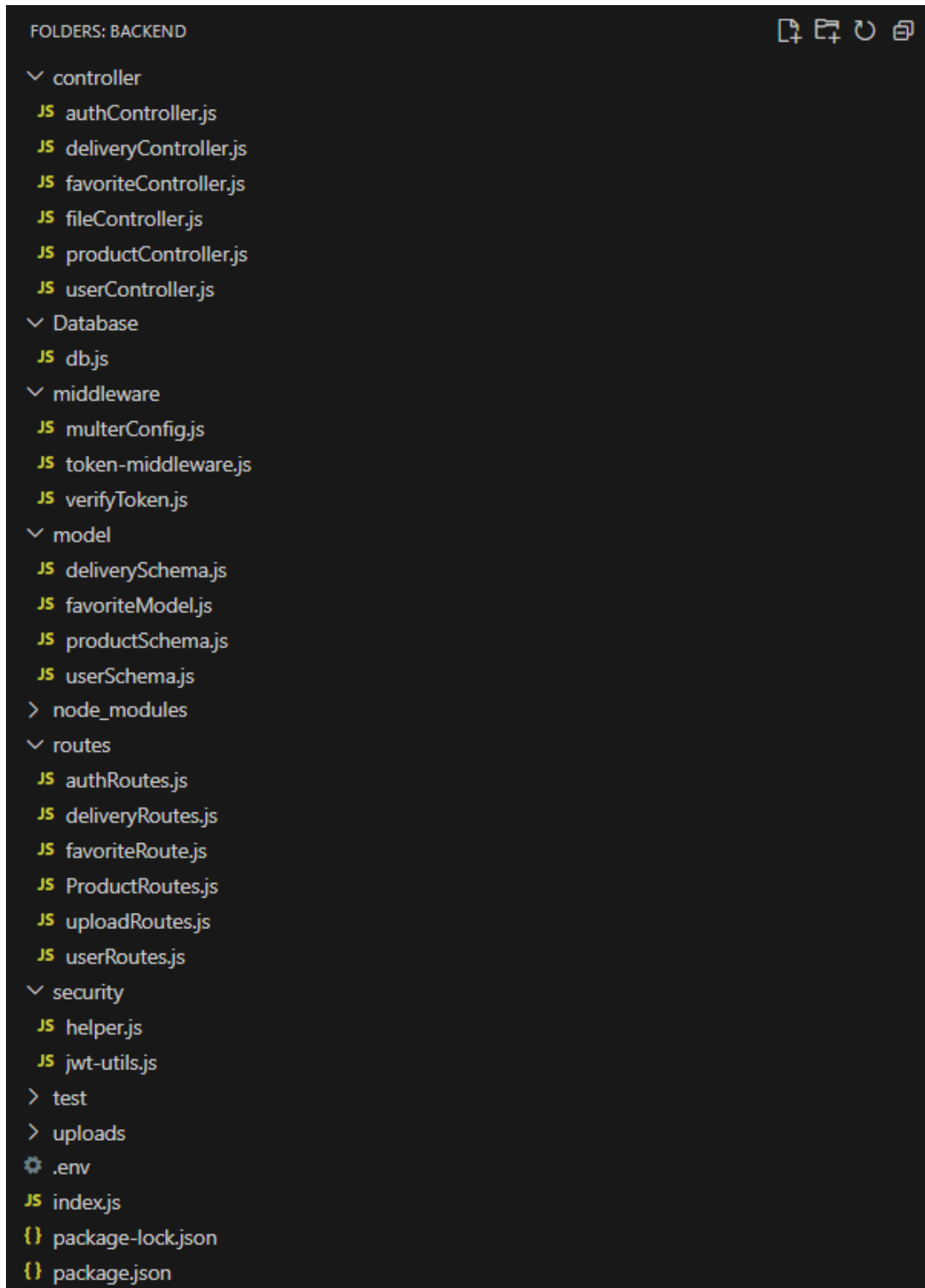


Figure 25: Backend Folder Structure

Database

Db.js

```
Database > JS db.js > ...
1  const { Sequelize } = require("sequelize");
2  require("dotenv").config();
3
4  const sequelize = new Sequelize(
5    process.env.DB_NAME,
6    process.env.DB_USER,
7    process.env.DB_PASS,
8    {
9      host: process.env.DB_HOST,
10     dialect: process.env.DB_DIALECT,
11   }
12 );
13
14 const connection = async () => {
15   try {
16     await sequelize.authenticate();
17     console.log("Connected to DB successfully.");
18     await sequelize.sync()
19     |   console.log("Connected to DB synced.");
20   } catch (error) {
21   }
22   console.error("Unable to connect:", error);
23 }
24 };
25
26 module.exports = { sequelize, connection };
27
```

Figure 26: Db.js code

Model

userSchema.js

```

model > JS userSchema.js > Users > constructor > email
1  const { DataTypes } = require("sequelize");
2  const { sequelize } = require("../Database/db");
3  const bcrypt = require("bcryptjs")
4
5  const Users = sequelize.define("users", {
6    name: {
7      type: DataTypes.STRING,
8      allowNull: false,
9    },
10   userId: {
11     type: DataTypes.INTEGER,
12     primaryKey: true,
13     autoIncrement: true
14   },
15   email: {
16     type: DataTypes.STRING,
17     allowNull: false,
18     unique: true,
19   },
20   phone: {
21     type: DataTypes.STRING,
22     allowNull: false,
23   },
24   password: {
25     type: DataTypes.STRING,
26     allowNull: false,
27   },
28   gender: {
29     type: DataTypes.STRING,
30     allowNull: false,
31   },
32 }, {
33   tableName: "users",
34
35   hooks: {
36     beforeCreate: async (user) => {
37       if (user.password) {

```

Figure 27: userSchema.js

```

model > JS userSchema.js > Users > constructor > email
5  const Users = sequelize.define("users", {
35    hooks: {
36      beforeCreate: async (user) => {
40        }
41      },
42      beforeUpdate: async (user) => {
43        if (user.changed('password')) {
44          const salt = await bcrypt.genSalt(10);
45          user.password = await bcrypt.hash(user.password, salt);
46        }
47      }
48    }
49  });
50
51  Users.prototype.comparePassword = async function(candidatePassword) {
52    return await bcrypt.compare(candidatePassword, this.password);
53  };
54
55  module.exports = Users;
56

```

Productschema.js

```

model > JS productSchema.js > Product
1  const { DataTypes } = require("sequelize");
2  const { sequelize } = require("../Database/db");
3
4  const Product = sequelize.define("Product", [
5
6    name: {
7      type: DataTypes.STRING,
8      allowNull: false,
9    },
10   image: {
11     type: DataTypes.STRING,
12     allowNull: true,
13   },
14   price: {
15     type: DataTypes.FLOAT,
16     allowNull: false,
17   },
18   quantity: {
19     type: DataTypes.INTEGER,
20     allowNull: false,
21   }
22 ], {
23
24   async () => {
25     try {
26       await Product.sync();
27       console.log(" Product table synced successfully!");
28     } catch (error) {
29       console.error("Error syncing table:", error);
30     }
31   })();
32
33   module.exports = Product;
34

```

Figure 28: productSchema.js

favoriteModel.js

```
model > JS favoriteModel.js > ...
1  const { DataTypes } = require("sequelize");
2  const { sequelize } = require("../Database/db");
3  const User = require("../userSchema");
4  const Product = require("../productSchema");
5
6  const Favorite = sequelize.define("Favorite", {
7    id: {
8      type: DataTypes.INTEGER,
9      autoIncrement: true,
10     primaryKey: true,
11   },
12   userId: {
13     type: DataTypes.INTEGER,
14     allowNull: false,
15   },
16   productId: {
17     type: DataTypes.INTEGER,
18     allowNull: false,
19   },
20 });
21
22 // OPTIONAL: Associations (if needed elsewhere)
23 User.hasMany(Favorite, { foreignKey: "userId" });
24 Product.hasMany(Favorite, { foreignKey: "productId" });
25 Favorite.belongsTo(User, { foreignKey: "userId" });
26 Favorite.belongsTo(Product, { foreignKey: "productId" });
27
28 module.exports = Favorite;
29
```

Figure 29: favoriteModel.js

DeliveySchema.js

```
model > JS deliverySchema.js > ...
1  const { DataTypes } = require('sequelize');
2  const { sequelize } = require('../Database/db');
3
4  const Delivery = sequelize.define('Delivery', {
5    firstName: {
6      type: DataTypes.STRING,
7      allowNull: false,
8    },
9    lastName: {
10     type: DataTypes.STRING,
11     allowNull: false,
12   },
13   phoneNumber: {
14     type: DataTypes.STRING,
15     allowNull: false,
16   },
17   email: {
18     type: DataTypes.STRING,
19     allowNull: false,
20   },
21   address: {
22     type: DataTypes.TEXT,
23     allowNull: false,
24   },
25   city: {
26     type: DataTypes.STRING,
27     allowNull: false,
28   },
29   paymentMethod: {
30     type: DataTypes.STRING,
31     allowNull: false,
32   },
33   instruction: {
34     type: DataTypes.TEXT,
35     allowNull: true,
36   },
37 });
```

Figure 30:deliverySchema.js

Controller

authController.js

```
controller > JS authController.js > ...
1  const Users = require("../model/userSchema"); // your Sequelize model
2  const { generateToken } = require("../security/jwt-utils");
3  const bcrypt = require("bcryptjs")
4
5  const login = async (req, res) => {
6    const { email, password } = req.body;
7
8    try {
9      if (!email || !password) {
10         return res.status(400).json({ message: "Email and password required." });
11      }
12
13      const user = await Users.findOne({ where: { email } });
14      const isPassword = await bcrypt.compare(password, user.password)
15
16      if (!isPassword) {
17         return res.status(401).json({ message: "Invalid credentials." });
18      }
19
20      const token = generateToken({ user: user.toJSON() });
21
22      res.status(200).json({
23         data: { access_token: token },
24         message: "Successfully logged in",
25      });
26    } catch (error) {
27      console.error("Login error:", error);
28      res.status(500).json({ message: "Server error during login" });
29    }
30  };
31
```

Figure 31: authController

controller > JS authController.js > ...

```

32
33 const register = async (req, res) => {
34
35     const { name, email } = req.body; //
36     console.log(req.body)
37
38     try {
39         const user = await Users.findOne({ where: { email } });
40         if (user == null) {
41             await Users.create(req.body);
42             return res.status(201).json({ message: "User added successfully" });
43         }
44         return res.status(400).json({ message: "User already exists" });
45     } catch (error) {
46         console.log(error);
47         return res.status(500).json({ message: "Error saving user" });
48     }
49 };
50 const init = async (req, res) => {
51     try {
52         const user = req.user.user;
53         delete user.password; // hide password in response
54
55         res.status(200).json({
56             data: user,
57             message: "Successfully fetched authenticated user",
58         });
59     } catch (error) {
60         console.error("Init error:", error);
61         res.status(500).json({ message: "Server error fetching user" });
62     }
63 };
64
65 module.exports = { login, init, register };

```

userController.js

```

controller > JS userController.js > [0] getAllEmployee
1  const Users = require("../model/userSchema");
2  const getAllEmployee = async (req, res) => {
3      console.log("Get Alls");
4
5      try {
6          const users = await Users.findAll();
7
8          return res.status(200).send({
9              data: users,
10             message: users.length === 0
11                 ? "No users found"
12                 : "Successfully fetched data"
13         });
14
15     } catch (error) {
16         console.error("Error while fetching:", error);
17         return res.status(500).json({ message: "Error while fetching data" });
18     }
19 };
20
21 const saveAllEmployee = async (req, res) => {
22     const { name, userId } = req.body; //
23     try {
24         const user = await Users.findOne({ where: { userId } });
25         if (user == null) {
26             await Users.create(req.body);
27             return res.status(201).json({ message: "User added successfully" });
28         }
29         return res.status(400).json({ message: "User already exists" });
30     } catch (error) {
31         console.log(error);
32         return res.status(500).json({ message: "Error saving user" });
33     }
34 };
35
36 module.exports = { getAllEmployee, saveAllEmployee };
37

```

Figure 32:userController.js

productController.js

```
controller > JS productController.js > ...
1  const Product = require("../model/productSchema");
2  // ADD PRODUCT
3  const addProduct = async (req, res) => {
4    try {
5      const { name, price, quantity } = req.body;
6      const image = req.file ? req.file.filename : null;
7
8      if (!name || !price || !quantity || !image) {
9        return res.status(400).json({ message: "All fields are required." });
10     }
11
12     const newProduct = await Product.create({ name, price, quantity, image });
13
14     return res.status(201).json({
15       message: "Product added successfully",
16       data: newProduct,
17     });
18   } catch (error) {
19     console.error("Add Product Error:", error);
20     res.status(500).json({ message: "Internal server error" });
21   }
22 };
23 // GET ALL PRODUCTS
24 const getAllProducts = async (req, res) => {
25   try {
26     const products = await Product.findAll();
27     return res.status(200).json(products);
28   } catch (error) {
29     console.error("Get Products Error:", error);
30     res.status(500).json({ message: "Internal server error" });
31   }
32 };
```

Figure 33: productController.js

```

controller > JS productController.js > ...
33 // DELETE PRODUCT
34 const deleteProduct = async (req, res) => {
35   try {
36     const id = req.params.id;
37     const deleted = await Product.destroy({ where: { id } });
38     if (!deleted) {
39       return res.status(404).json({ message: "Product not found" });
40     }
41     return res.status(200).json({ message: "Product deleted successfully" });
42   } catch (error) {
43     console.error("Delete Product Error:", error);
44     res.status(500).json({ message: "Internal server error" });
45   }
46 };
47
48 // ✅ FIXED: UPDATE PRODUCT
49 const updateProduct = async (req, res) => {
50   try {
51     const id = req.params.id;
52     const { name, price, quantity } = req.body;
53     const product = await Product.findByPk(id);
54
55     if (!product) {
56       return res.status(404).json({ message: "Product not found" });
57     }
58
59     const image = req.file ? req.file.filename : product.image;
60
61     await product.update({ name, price, quantity, image });
62
63     return res.status(200).json({
64       message: "Product updated successfully",
65       data: product,
66     });
67   } catch (error) {
68     console.error("Update Product Error:", error);
69     res.status(500).json({ message: "Internal server error" });
70   }
71 }
72
73 module.exports = {
74   addProduct,
75   getAllProducts,
76   deleteProduct,
77   updateProduct,
78 };
79

```

favoriteController.js

```

controller > JS favoriteController.js > [0] addToFavorite
1  const Favorite = require("../model/favoriteModel");
2  const Product = require("../model/productSchema");
3
4  const addToFavorite = async (req, res) => {
5      const userId = req.user.id; // from token middleware
6      const { productId } = req.body;
7      try {
8          // Check if product exists
9          const product = await Product.findByPk(productId);
10         if (!product) return res.status(404).json({ message: "Product not found" });
11
12         // Check if already favorited
13         const existing = await Favorite.findOne({ where: { userId, productId } });
14         if (existing) return res.status(400).json({ message: "Already favorited" });
15
16         // Add to favorites
17         await Favorite.create({ userId, productId });
18         res.status(201).json({ message: "Added to favorites" });
19     } catch (err) {
20         res.status(500).json({ error: err.message });
21     }
22 };
23
24 const getFavorites = async (req, res) => {
25     const userId = req.user.id;
26
27     try {
28         const favorites = await Favorite.findAll({
29             where: { userId },
30             include: [{ model: Product }],
31         });
32         res.status(200).json(favorites);
33     } catch (err) {
34         res.status(500).json({ error: err.message });
35     }
36 };
37

```

Figure 34: favoriteController.js

deliveryController.js

```

controller > JS deliveryController.js > ...
1  const Delivery = require('../model/deliverySchema');
2
3  const addDeliveryInfo = async (req, res) => {
4    try {
5      const {
6        firstName,
7        lastName,
8        phoneNumber,
9        email,
10       address,
11       city,
12       paymentMethod,
13       instruction
14     } = req.body;
15
16     const delivery = await Delivery.create({
17       firstName,
18       lastName,
19       phoneNumber,
20       email,
21       address,
22       city,
23       paymentMethod,
24       instruction,
25     });
26
27     res.status(201).json({ success: true, message: "Delivery Info Saved", delivery });
28   } catch (error) {
29     console.error("Delivery save error:", error);
30     res.status(500).json({ success: false, message: "Server Error", error });
31   }
32 };
33
34 module.exports = { addDeliveryInfo };
35

```

Figure 35: deliveryController.js

```

const removeFromFavorite = async (req, res) => {
  const userId = req.user.id;
  const { productId } = req.params;

  try {
    const deleted = await Favorite.destroy({ where: { userId, productId } });
    if (!deleted) {
      return res.status(404).json({ message: "Favorite not found" });
    }
    res.status(200).json({ message: "Removed from favorites" });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
};

module.exports = {
  addToFavorite,
  getFavorites,
  removeFromFavorite,
};

```

fileController.js

```
controller > JS fileController.js > ...
1  const uploadFile = (req, res) => {
2    try {
3      if (!req.file) {
4        return res.status(400).json({ message: "No file uploaded" });
5      }
6      res.status(200).json({
7        message: "File uploaded successfully!",
8        file: req.file,
9      });
10   } catch (error) {
11     res.status(500).json({
12       message: "Error uploading file",
13       error: error.message,
14     });
15   }
16 };
17
18 module.exports = { uploadFile };
19
20
```

Figure 36: fileController.js

Route

userRoute.js

```
routes > JS userRoutes.js > ...
1  const express = require("express");
2  const {
3    getAllEmployee,
4    saveAllEmployee,
5  } = require("../controller/userController");
6
7  const router = express.Router();
8
9  router.get("/", getAllEmployee);
10 router.post("/", saveAllEmployee);
11
12 module.exports = {router};
```

Figure 37: userRoute.js

authRoutes.js

```
routes > JS authRoutes.js > ...
1  const express = require("express");
2  const { login, init, register } = require("../controller/authController");
3
4  const router = express.Router();
5
6  router.post("/login", login);
7  router.post("/register", register); // POST /api/auth/register
8  router.get("/init", init);         // GET /api/auth/init (needs token)
9
10 module.exports = { authRouter: router };
```

Figure 38:authRoute.js

productRoutes.js

```
routes > JS ProductRoutes.js > ...
1  const express = require("express");
2  const upload = require("../middleware/multerConfig");
3  const {
4    getAllProducts,
5    addProduct,
6    deleteProduct,
7    updateProduct,
8  } = require("../controller/productController");
9
10 const router = express.Router();
11
12 router.get("/", getAllProducts);
13 router.post("/", upload.single("image"), addProduct);
14 router.delete("/:id", deleteProduct);
15 router.put("/:id", upload.single("image"), updateProduct); // Added multer middleware here
16
17 module.exports = { productRouter: router };
```

Figure 39:productRoute.js

favoriteRoutes.js

```

routes > JS favoriteRoute.js > ...
1  const express = require("express");
2  const router = express.Router();
3  const favoriteController = require("../controller/favoriteController");
4  const verifyToken = require("../middleware/verifyToken");
5
6  // Protect all favorite routes
7  router.post("/add", verifyToken, favoriteController.addToFavorite);
8  router.get("/", verifyToken, favoriteController.getFavorites);
9  router.delete("/:productId", verifyToken, favoriteController.removeFromFavorite);
10
11 module.exports = router;

```

Figure 40: favoriteRoute.js

deliveryRoutes.js

```

routes > JS deliveryRoutes.js > ...
1  const express = require('express');
2  const router = express.Router();
3  const { addDeliveryInfo } = require('../controller/deliveryController');
4
5  router.post('/', addDeliveryInfo);
6
7  module.exports = router;

```

Figure 41: deliveryRoute.js

uploadRoutes

```

1  const express = require("express");
2  const upload = require("../middleware/multerConfig");
3  const { uploadFile } = require("../controller/fileController");
4
5  const router = express.Router();
6
7  router.post("/upload", upload.single("file"), uploadFile);
8
9  module.exports = router; // ✅ Export the actual router

```

Figure 42: uploadRoutes.js

Middleware

multerConfig.js

```
middleware > JS multerConfig.js > ...
1  const multer = require("multer");
2  const path = require("path");
3
4  // Storage config
5  const storage = multer.diskStorage({
6    destination: (req, file, cb) => {
7      cb(null, "uploads/");
8    },
9    filename: (req, file, cb) => {
10     cb(null, Date.now() + path.extname(file.originalname));
11   },
12 });
13
14 // File filter
15 const fileFilter = (req, file, cb) => {
16   const allowedTypes = ["image/jpeg", "image/png", "image/gif"];
17   if (allowedTypes.includes(file.mimetype)) {
18     cb(null, true);
19   } else {
20     cb(new Error("Invalid file type. Only JPEG, PNG, and GIF are allowed!"), false);
21   }
22 };
23
24 // Multer instance
25 const upload = multer({
26   storage,
27   fileFilter,
28   limits: { fileSize: 5 * 1024 * 1024 }, // 5MB
29 });
30
31 module.exports = upload; // ✅ DO THIS – not export default
32
```

Figure 43:multerConfig.js

token-middleware.js

```

middleware > JS token-middleware.js > authenticateToken > jwt.verify() callback
1  const jwt = require("jsonwebtoken");
2  const Users = require("../model/userSchema");
3  require("dotenv").config();
4
5  const authenticateToken = (req, res, next) => {
6    // Allow login and signup without token
7    if (
8      req.path === "/api/auth/login" ||
9      (req.path === "/api/auth/register" && (req.method === "POST" || req.method === "GET")) || "api/Users"
10   ) {
11     return next();
12   }
13
14   const authHeader = req.headers["authorization"];
15   const token = authHeader && authHeader.split(" ")[1];
16
17   if (!token) {
18     return res.status(401).json({ message: "Access denied. No token provided." });
19   }
20
21   jwt.verify(token, process.env.JWT_SECRET, (err, decoded) => {
22     if (err) {
23       return res.status(403).json({ message: "Invalid or expired token." });
24     }
25
26     req.user = decoded;
27     next();
28   });
29 };
30
31 module.exports = { authenticateToken };
32

```

Figure 44: token-middleware.js

Verifytoken.js

```

middleware > JS verifyToken.js > ...
1  const jwt = require("jsonwebtoken");
2
3  const verifyToken = (req, res, next) => {
4      const token = req.headers.authorization?.split(" ")[1];
5      if (!token) return res.status(401).json({ message: "Access denied" });
6
7      try {
8          const decoded = jwt.verify(token, process.env.JWT_SECRET);
9          req.user = decoded;
10         next();
11     } catch (err) {
12         res.status(403).json({ message: "Invalid token" });
13     }
14 };
15
16 module.exports = verifyToken;

```

Figure 45:verifyToken.js

Security

helper.js

```

security > JS helper.js > ...
1  import fs from "fs";
2
3  const createUploadsFolder = () => {
4      const dir = "./uploads";
5      if (!fs.existsSync(dir)) {
6          fs.mkdirSync(dir);
7          console.log("Uploads folder created.");
8      }
9  };
10
11 export { createUploadsFolder };

```

Figure 46:helper.js

Jwt-utils.js

```

security > JS jwt-utils.js > ...
1  const jwt = require("jsonwebtoken");           // Import JWT package
2  require("dotenv").config();                     // Load .env variables
3
4  const generateToken = (payload) => {
5      return jwt.sign(payload, process.env.JWT_SECRET, { expiresIn: "1h" });
6      // Sign the payload with the secret key and set token to expire in 1 hour
7  };
8
9  module.exports = { generateToken };             // Export for use in controller

```

Figure 47: jwt-utils.js

Index.js

```

const express = require("express");
const cors = require("cors");
const dotenv = require("dotenv");
const { connection } = require("./Database/db");
const { router: userRouter } = require("./routes/userRoutes");
const { authRouter } = require("./routes/authRoutes");
const authenticateToken = require("./middleware/token-middleware");
const uploadRouter = require("./routes/uploadRoutes");
const { createUploadsFolder } = require("./security/helper");
const { productRouter } = require("./routes/ProductRoutes");
const multer = require("multer");
const Favorite = require("./model/favoriteModel");
const deliveryRouter = require("./routes/deliveryRoutes");

const { sequelize } = require("./Database/db");
sequelize.sync().then(() => {
    console.log("All models synced successfully.");
});

dotenv.config();

const app = express();
const PORT = process.env.PORT || 5000;

app.use(cors({
    origin: "http://localhost:5173",
    credentials: true
}));

```

Figure 48:index.js

```

dotenv.config();

const app = express();
const PORT = process.env.PORT || 5000;

app.use(cors({
  origin: "http://localhost:5173",
  credentials: true
}));

app.use(express.json());

const favoriteRoutes = require("./routes/favoriteRoute");
app.use("/api/favorites", favoriteRoutes);

// Create uploads folder at startup
createUploadsFolder();

app.use('/api/delivery', deliveryRouter);
|

// ✅ Serve uploaded files statically
app.use("/uploads", express.static("uploads"));

// ✅ Protected routes
app.use("/api/users", authenticateToken, userRouter); // 🔒 Only protect sensitive routes

// ✅ Routes

app.use("/api/auth", authRouter);
app.use("/api/file", uploadRouter);
app.use("/api/products", productRouter);

```

```

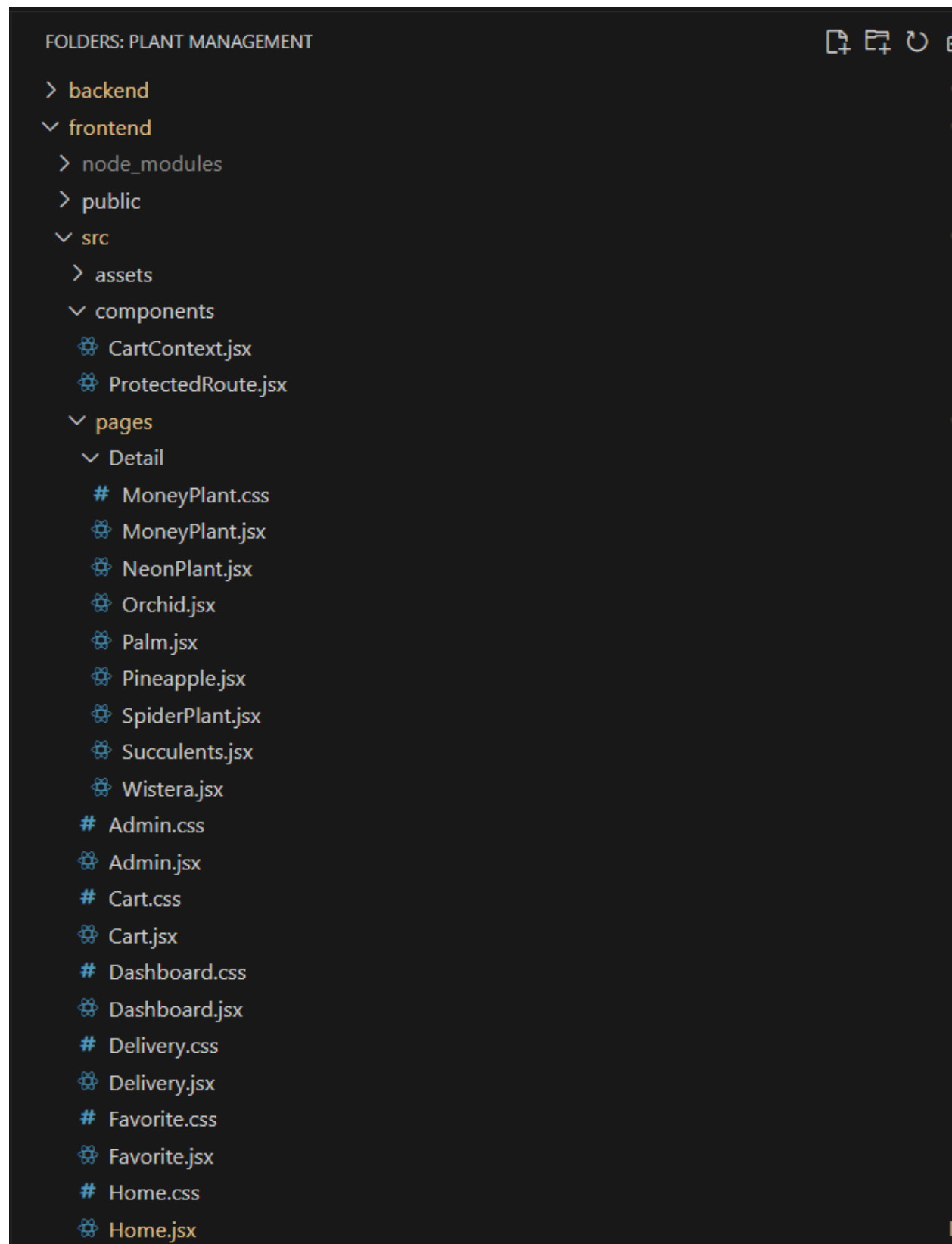
// *** NEW: Multer error handling middleware ***
app.use((err, req, res, next) => {
  if (err instanceof multer.MulterError) {
    return res.status(400).json({ message: err.message });
  }
  next(err);
});

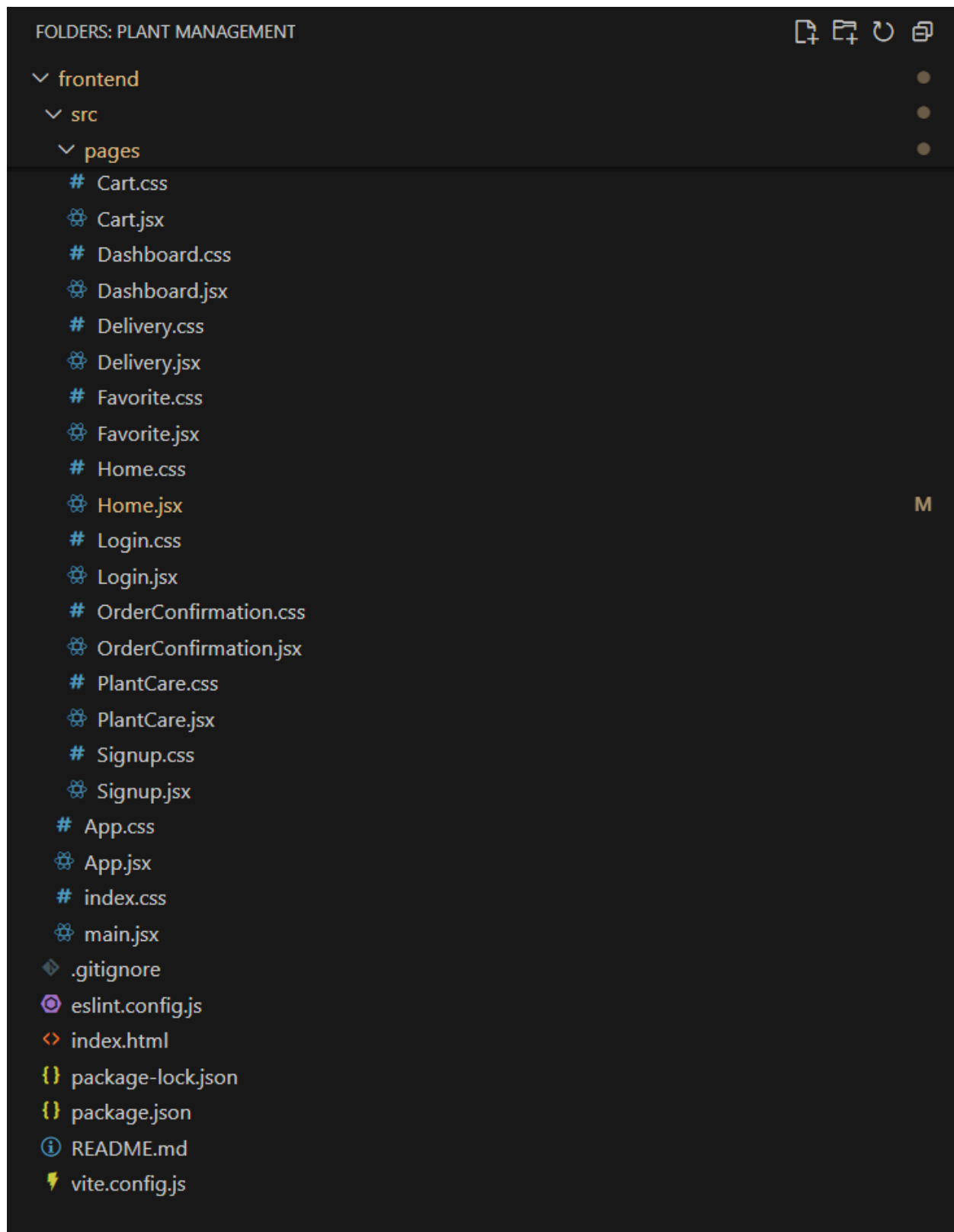
// ✅ Connect to DB
connection();

// ✅ Start server
app.listen(PORT, () => {
  console.log(`Server running on port ${PORT}`);
});

```

Frontend folder Structure





Home.jsx

```

import React, { useState } from 'react';
import { useNavigate } from 'react-router-dom';
import { Link } from 'react-router-dom';
import { assets } from '../assets/assets';
import Swal from 'sweetalert2';
import './Home.css';

const Home = () => {
  const [menu, setMenu] = useState("home");
  const navigate = useNavigate();

  return (
    <div>
      <div className='navbar'>
        <img src={assets.final_logo} alt="" className='logo' />
        <ul className='navbar-menu'>
          <li
            onClick={() => setMenu("home")}
            className={menu === "home" ? "active" : ""}
          >
            Home
          </li>
          <li
            onClick={() => {
              setMenu("outlet");
              Swal.fire({
                title: 'Our Outlets 🌱',
                text: 'Explore our nearest plant outlets across the city!',
                icon: 'info',
                confirmButtonColor: '#66c35c',
              });
            }}
            className={menu === "outlet" ? "active" : ""}
          >
            Outlets
          </li>
        </ul>
      </div>
    </div>
  );
};

```

Figure 49: Home.jsx

```

    <div className='navbar-right'>
      <button onClick={() => navigate('/login')}>Login</button>
      <button onClick={() => navigate('/signup')}>Register</button>
    </div>
  </div>

  <section className='main-img'>
    <img src={assets.plant} alt="" className="background-img" />
    <div className="top-leftText">
      <h1>Fresh plants delivered To Your <span className="door">Door</span> <span className="step">Step</span></h1>
    </div>

    <div className="top-leftText2">
      <p>"Bring nature home with handpicked plants delivered to your door. Fast, fresh, and hassle-free!"</p>
    </div>
  </section>

  <section className="categories-section">
    <h3>Explore Categories</h3>
    <p>Discover food from your favorite cuisines</p>

    <div className="categories">
      <div className="category-card">
        <img src={assets.delivery_icon} />
        <span>Fast Delivery</span>
      </div>
      <div className="category-card">
        <img src={assets.service_icon} />
        <span>24/7 Support</span>
      </div>
      <div className="category-card">
        <img src={assets.plant_icon} />
        <span>Plant care</span>
      </div>
    </div>

    <div className="category-card">
      <img src={assets.plant_icon} />
      <span>Plant care</span>
    </div>
    <div className="category-card">
      <img src={assets.payment_icon} />
      <span>Secure Payment</span>
    </div>
    <div className="category-card">
      <img src={assets.gift_icon} />
      <span>More</span>
    </div>
  </div>
</section>
</div>
);
};

export default Home;

```

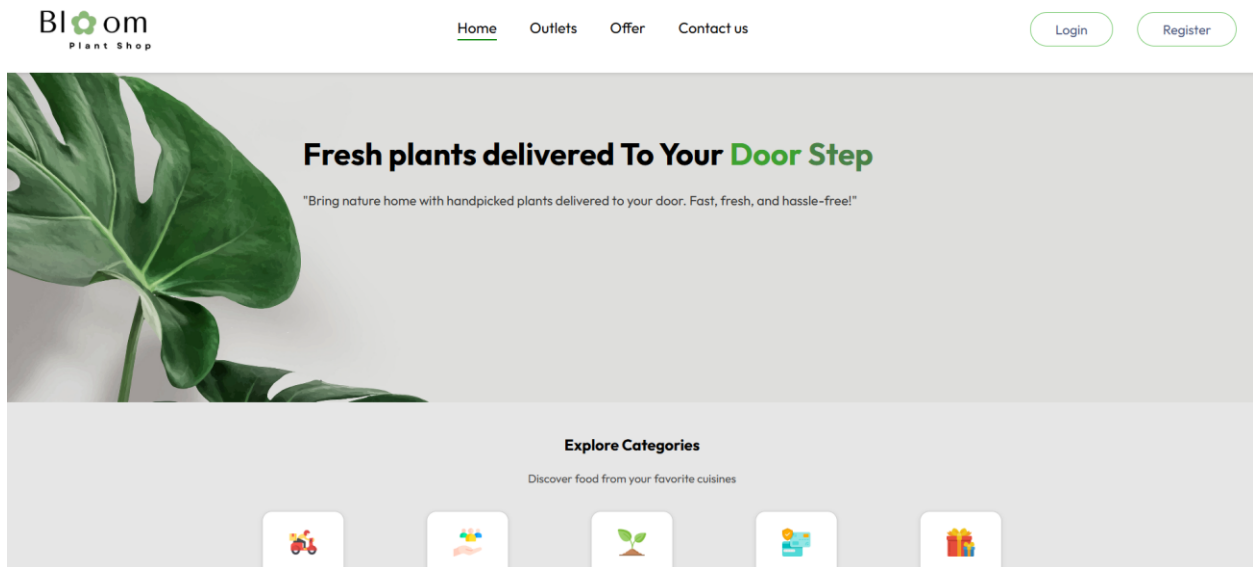


Figure 50: Home frontend

Login.jsx

```
import React from 'react';
import { useForm } from 'react-hook-form';
import axios from 'axios';
import { useNavigate, Link } from 'react-router-dom';
import { assets } from '../assets/assets';
import './Login.css';

const Login = () => {
  const { register, handleSubmit } = useForm();
  const navigate = useNavigate();

  const onSubmit = async (data) => {
    try {
      const res = await axios.post('http://localhost:5000/api/auth/login', {
        email: data.email,
        password: data.password,
      });

      console.log(res.data.data.access_token)
      // Save token in localStorage
      localStorage.setItem('token', res.data.data.access_token);

      alert('Login successfull!');
      navigate('/dashboard'); // redirect to a protected route or home page
    } catch (error) {
      alert(error.response?.data?.message || 'Login failed');
    }
  };
};
```

Figure 51:login.jsx

```

return (
  <div>
    <div className="navbar">
      <img src={assets.final_logo} alt="" className="logo" />
      <ul className="navbar-menu1">
        <li><Link to="/">Home</Link></li>
        <li>Outlets</li>
        <li>Offer</li>
        <li>Contact us</li>
      </ul>
    </div>

    <div className="container">
      <div className="form-box">
        <h3>Sign in to your account</h3><br />
        <form onSubmit={handleSubmit(onSubmit)}>
          <div className="form-group">
            <label>Email</label>
            <input type="email" {...register('email', { required: true })} placeholder="abc@gmail.com" />
          </div>

          <div className="form-group">
            <label>Password</label>
            <input type="password" {...register('password', { required: true })} placeholder="*****" />
            <div className="forgot-password">
              <a href="#">Forgot password?</a>
            </div>
          </div>

          <div className="btn-container">
            <button className="register-btn" type="submit">Sign in</button>
          </div>

          <div className="register-link">
            Don't have an account? <Link to="/signup">Register here</Link>
          </div>

          <input type="password" {...register('password', { required: true })} placeholder="*****" />
          <div className="forgot-password">
            <a href="#">Forgot password?</a>
          </div>
        </form>

        <div className="btn-container">
          <button className="register-btn" type="submit">Sign in</button>
        </div>

        <div className="register-link">
          Don't have an account? <Link to="/signup">Register here</Link>
        </div>
      </div>
    </div>
  </div>
);
};

export default Login;

```

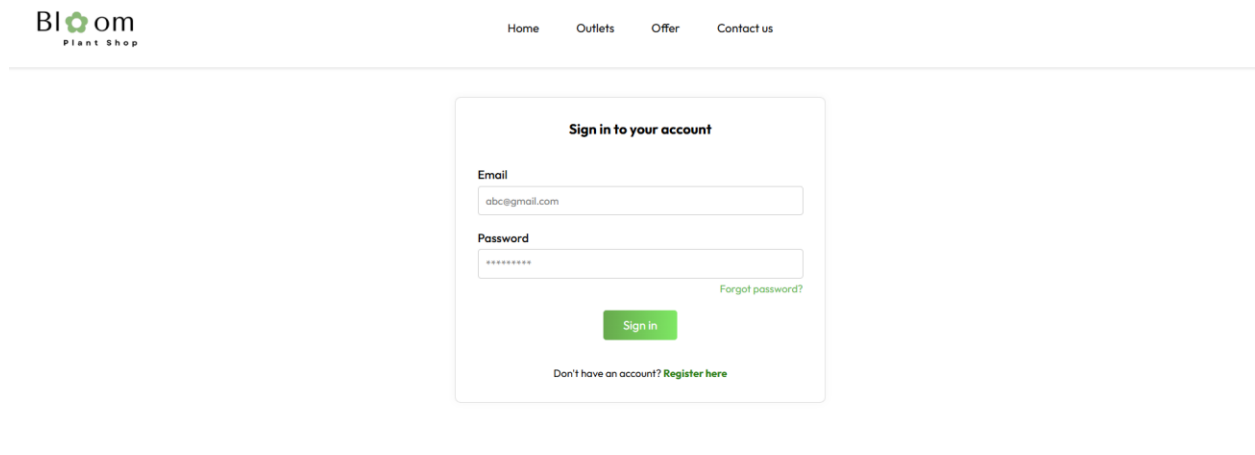


Figure 52: Login frontend

Signup.jsx

```

import React from 'react';
import { useForm } from 'react-hook-form';
import axios from 'axios';
import { Link } from 'react-router-dom';
import { assets } from '../assets/assets';
import './Signup.css';
const Signup = () => {
  const {
    register,
    handleSubmit,
    watch,
    reset,
    formState: { errors }
  } = useForm();

  const onSubmit = async (data) => {
    if (data.password !== data.confirmPassword) {
      alert("Passwords do not match.");
      return;
    }

    try {
      const res = await axios.post("http://localhost:5000/api/auth/register", {
        name: data.name,
        userId: data.userId,
        email: data.email,
        phone: data.phone,
        password: data.password,
        gender: data.gender,
      });
    }
  }
}

```

Figure 53: Signup.jsx



[Home](#)
[Outlets](#)
[Offer](#)
[Contact us](#)

Registration

Full Name

Email

Phone Number

Password

Confirm Password

Gender:
☐ Male
 ☐ Female
 ☐ Other

Already have an account? [Login](#)

Figure 54: Signup frontend

Admin.jsx

```

import React, { useState, useEffect } from "react";
import { useNavigate } from "react-router-dom";
import axios from "axios";
import './Admin.css';
import { assets } from '../assets/assets';

const Admin = () => {
  const [name, setName] = useState("");
  const [price, setPrice] = useState("");
  const [quantity, setQuantity] = useState("");
  const [image, setImage] = useState(null);
  const [products, setProducts] = useState([]);
  const [editId, setEditId] = useState(null);
  const navigate = useNavigate();

  const fetchProducts = async () => {
    try {
      const res = await axios.get("http://localhost:5000/api/products");
      setProducts(res.data);
    } catch (err) {
      console.error("Failed to fetch products:", err);
    }
  };
};

```

Figure 55: Admin.jsx

```

useEffect(() => {
  fetchProducts();
}, []);

const handleSubmit = async (e) => {
  e.preventDefault();

  const formData = new FormData();
  formData.append("name", name);
  formData.append("price", price);
  formData.append("quantity", quantity);
  if (image) formData.append("image", image);

  try {
    if (editId) {
      await axios.put(`http://localhost:5000/api/products/${editId}`, formData, {
        headers: { "Content-Type": "multipart/form-data" },
      });
      alert("✏ Product updated successfully!");
    } else {
      await axios.post("http://localhost:5000/api/products", formData, {
        headers: { "Content-Type": "multipart/form-data" },
      });
      alert("✅ Product added successfully!");
    }

    setName("");
    setPrice("");
    setQuantity("");
    setImage(null);
    setEditId(null);
    fetchProducts();
  } catch (err) {
    console.error("❌ Error:", err.response?.data || err.message);
    alert("Something went wrong");
  }
};

```

```

const handleDelete = async (id) => {
  if (!window.confirm("Are you sure you want to delete this product?")) return;

  try {
    await axios.delete(`http://localhost:5000/api/products/${id}`);
    alert("🗑️ Product deleted successfully!");
    fetchProducts();
  } catch (err) {
    console.error("❌ Delete failed:", err);
    alert("Failed to delete product.");
  }
};

const handleEdit = (product) => {
  setName(product.name);
  setPrice(product.price);
  setQuantity(product.quantity);
  setImage(null); // new image optional
  setEditId(product.id);
};

const handleLogout = () => {
  localStorage.removeItem("token");
  navigate("/");
};

return (
  <div className="admin-container">
    <aside className="admin-sidebar">
      <div className="admin-logo">
        <img src={assets.final_logo} alt="Bloom Logo" />
      </div>
      <h2>Admin Panel</h2>

      <form className="admin-form" onSubmit={handleSubmit} encType="multipart/form-data">
        <label>Name</label>
        <input
          type="text"
          placeholder="Product Name"
          value={name}
          onChange={(e) => setName(e.target.value)}
          required
        />
      </form>
    </aside>
  </div>
)

```

```

    {product.image}&`"
    alt={product.name}
    height="40"
  />
</td>
<td>{product.price}</td>
<td>{product.quantity}</td>
<td>
    <img
      src={assets.updated}
      className="admin-icon"
      alt="Update"
      onClick={() => handleEdit(product)}
    />
    <img
      src={assets.trash}
      className="admin-icon"
      alt="Delete"
      onClick={() => handleDelete(product.id)}
    />
  </td>
</tr>
</tbody>
</table>

  { /* Logout Icon */ }
  <div className="admin-logout">
    <img
      src={assets.logout}
      alt="Logout"
      className="logout-icon"
      onClick={handleLogout}
    />
  </div>
</main>
</div>
);
};

export default Admin;

```

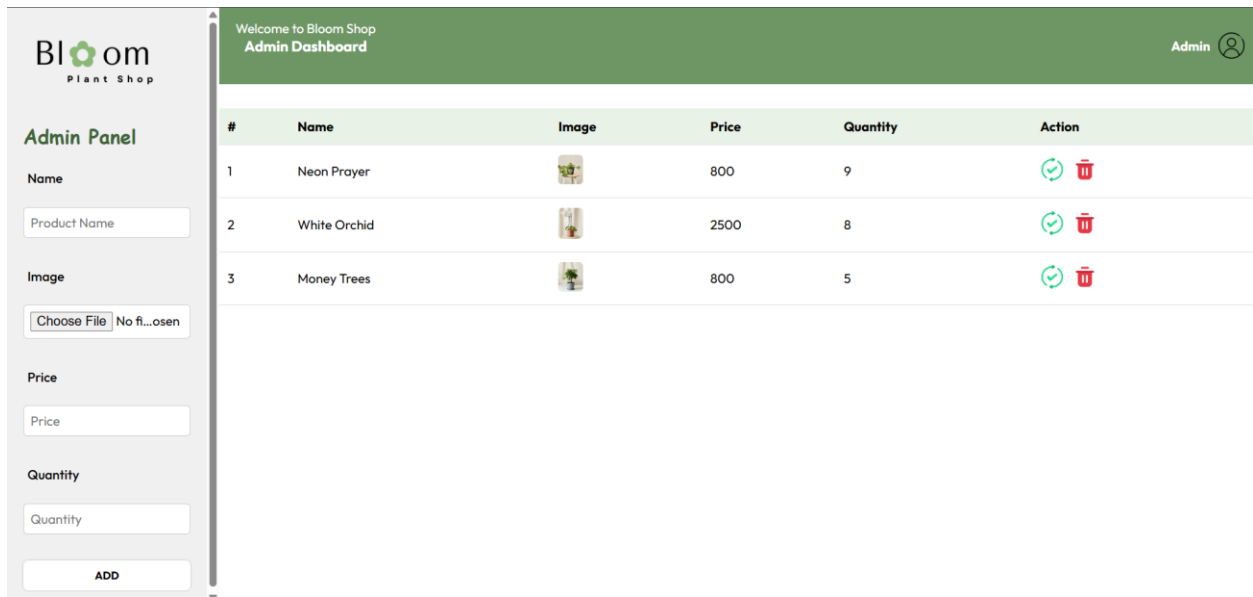


Figure 56: Admin frontend

Delivery.jsx

```
import React from 'react';
import { assets } from '../assets/assets';
import './Delivery.css';
import { useNavigate } from 'react-router-dom';
import { useForm } from 'react-hook-form';
import axios from 'axios';

const Delivery = () => {
  const navigate = useNavigate();

  const {
    register,
    handleSubmit,
    formState: { errors, isSubmitting },
  } = useForm();

  const onSubmit = async (data) => {
    try {
      await axios.post('http://localhost:5000/api/delivery', data);
      navigate('/confirm');
    } catch (error) {
      console.error('Failed to save delivery info:', error);
      alert('Failed to save delivery information. Please try again.');
    }
  };
};
```

```

return (
  <div>
    <div className="checkout-container">
      { /* LEFT SIDE: Only this part is a form */ }
      <form
        onSubmit={handleSubmit(onSubmit)}
        className="checkout-form-section padded-section"
        style={{ maxWidth: "50%" }}
      >
        <div className="checkout-section-header">
          <img src={assets.location} alt="location icon" />
          Delivery Information
        </div>

        <div className="checkout-form-group">
          <label>First Name</label>
          <input
            type="text"
            {...register('firstName', { required: 'First Name is required' })}
          />
          {errors.firstName && <p className="form-error">{errors.firstName.message}</p>}
        </div>

        <div className="checkout-form-group">
          <label>Last Name</label>
          <input
            type="text"
            {...register('lastName', { required: 'Last Name is required' })}
          />
          {errors.lastName && <p className="form-error">{errors.lastName.message}</p>}
        </div>

        <div className="checkout-form-group">
          <label>Phone Number</label>
          <input
            type="tel"
            {...register('phoneNumber', { required: 'Phone Number is required' })}
          />
          {errors.phoneNumber && <p className="form-error">{errors.phoneNumber.message}</p>}
        </div>
      </form>
    </div>
  </div>
)

```

```

<div className="checkout-form-group">
  <label>Complete Address</label>
  <textarea {...register('address', { required: 'Address is required' })}></textarea>
  {errors.address && <p className="form-error">{errors.address.message}</p>}
</div>

<div className="checkout-form-group">
  <label>City</label>
  <select {...register('city', { required: 'City is required' })}>
    <option value="">Select a city</option>
    <option value="Kathmandu">Kathmandu</option>
    <option value="Pokhara">Pokhara</option>
  </select>
  {errors.city && <p className="form-error">{errors.city.message}</p>}
</div>

<div className="checkout-form-group">
  <label>Payment Method</label>
  <button
    type="button"
    className="checkout-payment-method"
    tabIndex="0"
    onClick={() => alert('Only Cash on Delivery accepted for now')}
  >
    <img src={assets.cod} alt="cash on delivery" />
    <div className="checkout-payment-method-text">Cash on Delivery (COD)</div>
  </button>
  <input
    type="hidden"
    value="Cash on Delivery"
    {...register('paymentMethod', { required: true })}
  />
  {errors.paymentMethod && <p className="form-error">Payment Method is required</p>}
</div>

<button
  type="submit"
  disabled={isSubmitting}
  className="checkout-place-order-btn"
>
  {isSubmitting ? 'Placing Order...' : 'PLACE ORDER'}
</button>
</form>

```

```

{ /* RIGHT SIDE: Outside the form to keep flex layout */}
<div className="checkout-summary-section padded-section" style={{ maxWidth: "40%" }}>
  <div className="checkout-form-group special-instructions-group">
    <label><strong>Special Instructions (Optional)</strong></label>
    <textarea {...register('instruction')}></textarea>
  </div>

  <div className="checkout-order-summary">
    <h3>
      <img src={assets.bill} alt="bill icon" />
      Order Summary
    </h3>

    <div className="checkout-item">
      <div>Orchid</div>
      <div>x1</div>
      <div className="checkout-price">Rs. 1500</div>
    </div>

    <div className="checkout-totals">
      <div>
        <span>Subtotal</span>
        <span>Rs. 1500</span>
      </div>
      <div>
        <span>Delivery Fee</span>
        <span>Rs. 120</span>
      </div>
      <div className="checkout-total">
        <span>Total:</span>
        <span>Rs. 1700</span>
      </div>
    </div>
  </div>
</div>
</div>
);
};

export default Delivery;

```


Delivery Information

First Name

Last Name

Phone Number

Email Address

Complete Address

City

Special Instructions (Optional)


Order Summary

Orchid	x1	Rs. 1500
Subtotal		Rs. 1500
Delivery Fee		Rs. 120
Total:		Rs. 1700

Figure 57: Delivery Frontend

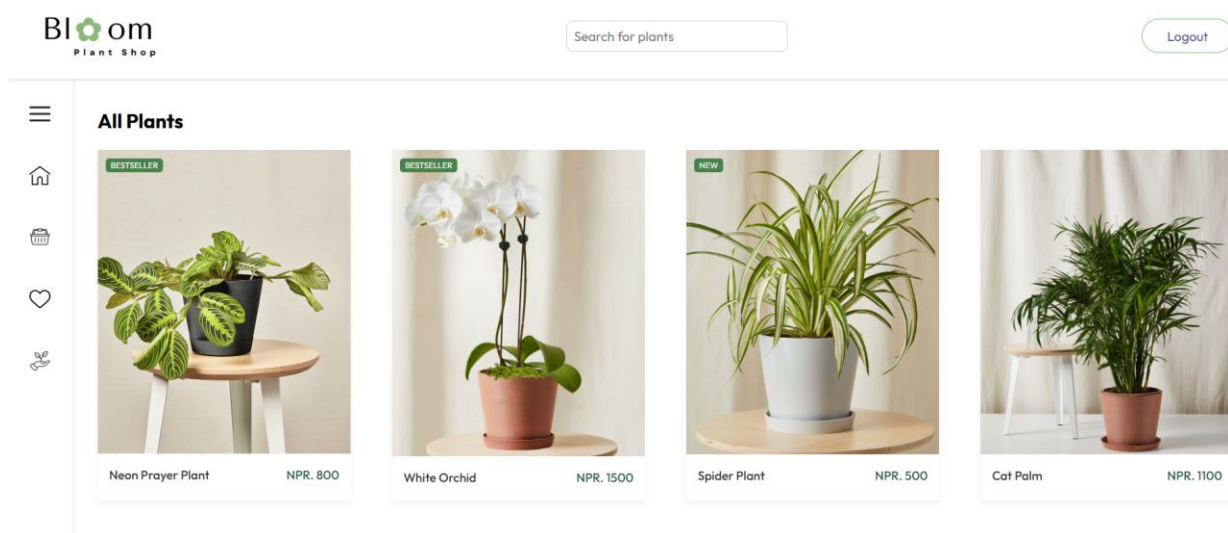


Figure 58: User Dashboard frontend

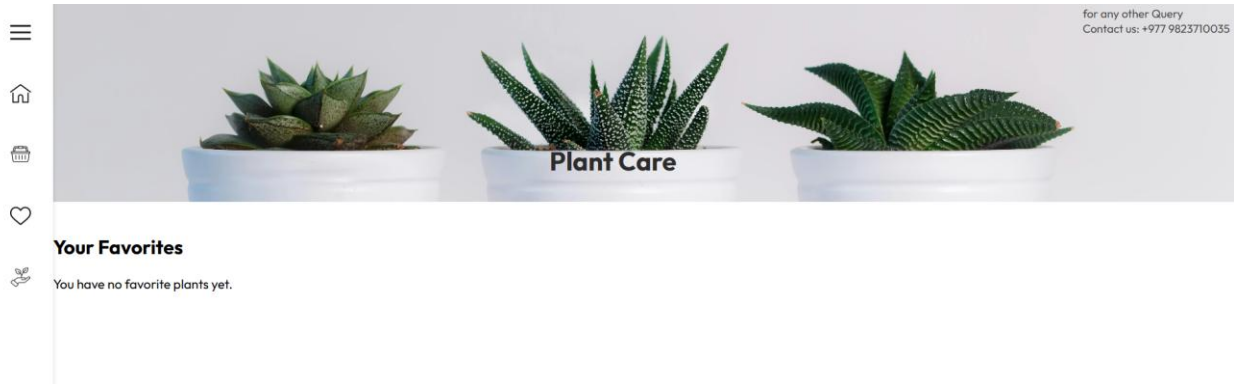


Figure 59: Favorite Frontend

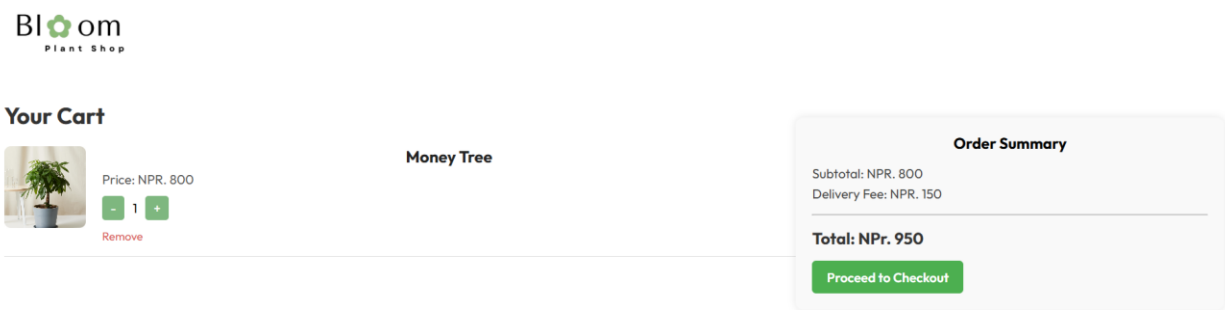


Figure 60: Cart Frontend

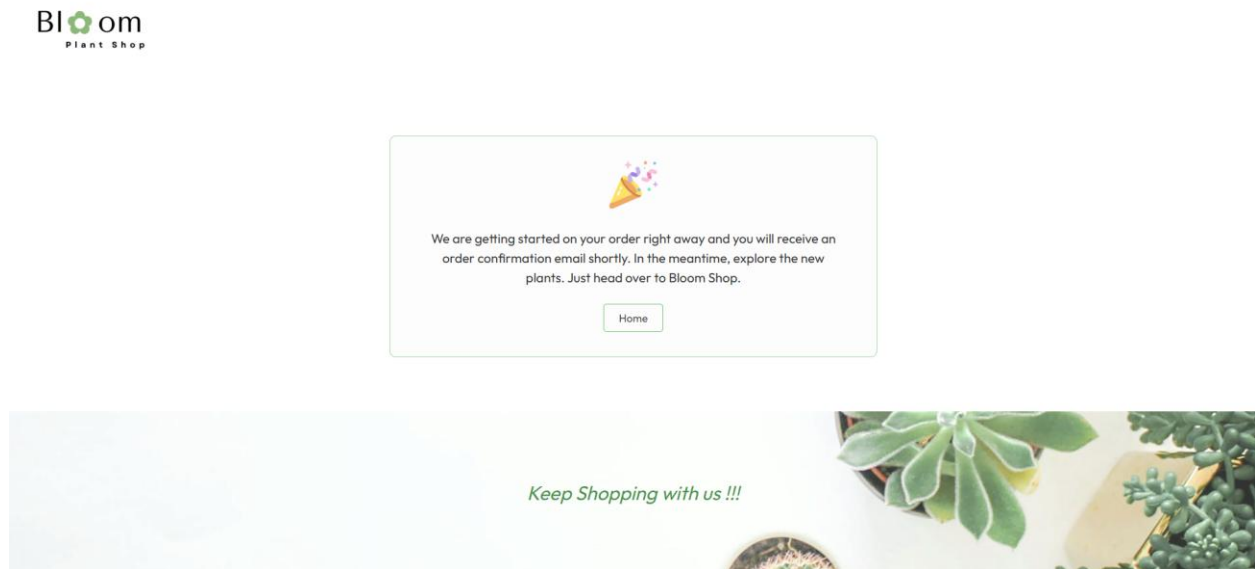


Figure 61: Order Confirmation frontend

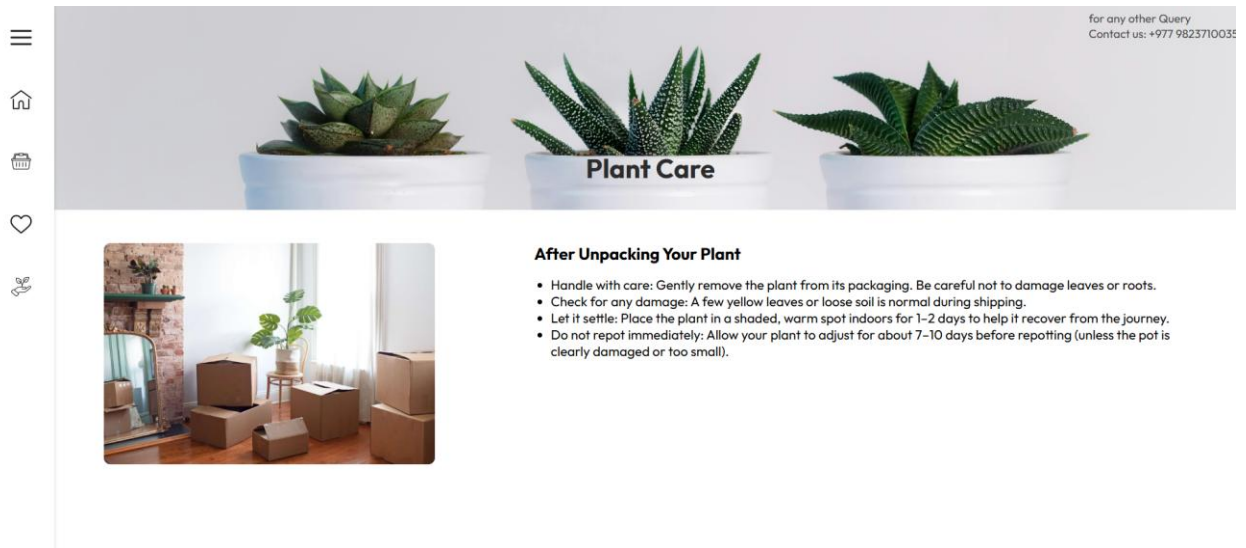


Figure 62: Plant Care Fronend

Database

Table

- ✓  Tables (5)
- >  Deliveries
 - >  Favorites
 - >  Products
 - >  Users
 - >  users

user table

1 **SELECT** * **FROM** public.users
2 **ORDER BY** "userId" **ASC**

Data Output Messages Notifications

Showing rows: 1 to 3 Page No: 1 of 1

	name character varying (255)	userId [PK] integer	email character varying (255)	phone character varying (255)	password character varying (255)	gender character varying (255)	createdAt timestamp with time
1	Test User	1	testuser@example.com	9876543210	\$2b\$10\$52nGFafC2u0.hEh4TSid0uqvVHHvtZD1C6PkGKmHhRxQpadt...	female	2025-08-01 09:56:1
2	Deepesh	2	deepesh@gmail.com	9851009530	\$2b\$10\$vKCragYrtLI6CeIUH/Fipu7o6t9frEJqZZ9KWLUPQ2Getft87sLV6	Male	2025-08-01 12:42:5
3	Anisha G.C	3	anisha@gmail.com	9851009530	\$2b\$10\$r98z9VegfcVDD65QfMieezkHDBs7T/Dxe5O2sludf.cWRjt0s746	Female	2025-08-01 12:45:0

Figure 63: user table

Delivery table

Query Query History

1 **SELECT** * **FROM** public."Deliveries"
2 **ORDER BY** id **ASC**

Scratch Pad

Data Output Messages Notifications

Showing rows: 1 to 3 Page No: 1 of 1

	id [PK] integer	firstName character varying (255)	lastName character varying (255)	phoneNumber character varying (255)	email character varying (255)	address text	city character varying (255)	paymentMethod character varying (255)
1	1	anisha	gc	9851009530	gcanisha4@gmail.com	K.M Korean language , Gongabu Chowk	Kathmandu	Cash on Delivery
2	2	Ninam	rai	98563214	ninamrai@gmail.com	K.M Korean language , Gongabu Chowk	Pokhara	Cash on Delivery
3	3	Anisha	GC	985100950	anisha@gmail.com	Tokha-11,Samakhusi Chowk	Kathmandu	Cash on Delivery

Figure 64: delivery table

Product table

Query Query History

Scratch Pad X

1

select * from "Products"

Data Output Messages Notifications

SQL

Showing rows: 1 to 3

Page No: 1

	id [PK] integer	name character varying (255)	image character varying (255)	price double precision	quantity integer	createdAt timestamp with time zone	updatedAt timestamp with time zone
1	2	Neon Prayer	1753271888234.png	800	9	2025-07-23 17:43:08.247+05:45	2025-07-24 22:41:53.159+05:45
2	3	White Orchid	1753272941944.png	2500	8	2025-07-23 18:00:41.952+05:45	2025-07-24 22:42:27.679+05:45
3	4	Money Trees	1753368687784.jpg	800	5	2025-07-24 20:36:27.789+05:45	2025-07-29 13:41:52.665+05:45

Figure 65: product table

Testing and Evaluation

The Testing and Evaluation phase was crucial to ensuring the Online Plant Shopping System functioned reliably and met the intended user requirements. This phase helped identify and fix bugs early, resulting in a stable, secure, and user-friendly web application. Various testing strategies were employed throughout development:

→Unit Testing was implemented using Jest to test individual backend functions, such as user registration, product CRUD operations, and authentication. These tests verified that each module performed as expected in isolation.

→Integration Testing with Supertest ensured that the backend APIs functioned correctly when connected with routes, middleware, and the database. Key areas tested included login, registration, and product listing.

→Manual Testing was done on the frontend using React. This included checking the functionality of the login/signup forms, product display pages, add-to-favorites, and admin dashboard actions like adding and deleting products.

→User Acceptance Testing (UAT) was conducted among team members and peers to simulate real user behavior. Feedback helped refine the UI/UX and fix usability issues, especially around navigation and responsiveness.

→Evaluation was performed by analyzing performance, security, and functionality. JWT was used for secure authentication, while form validations and error handling improved reliability. Users were able to interact with the system smoothly, confirming it met usability expectations.

Testing and Evaluation

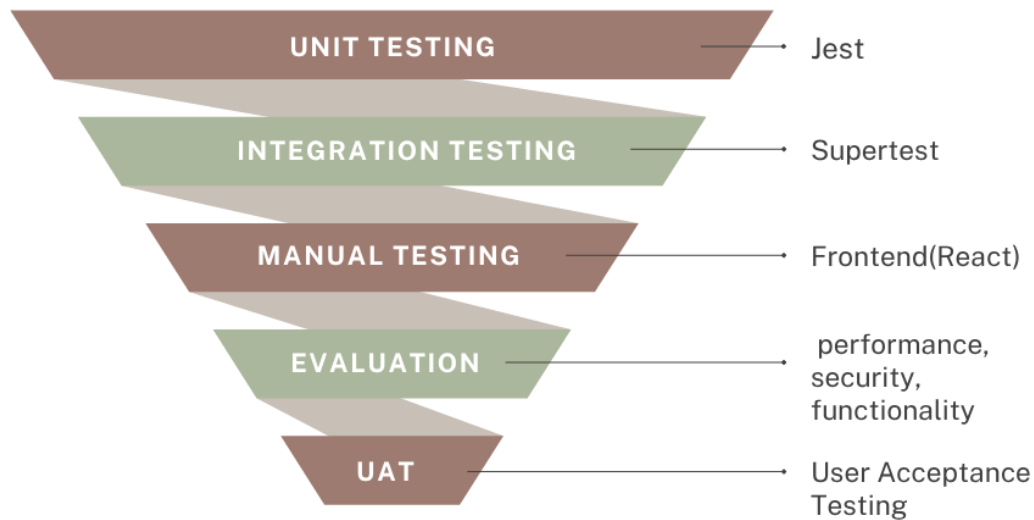


Figure 66: Testing and Evaluation

Unit Testing

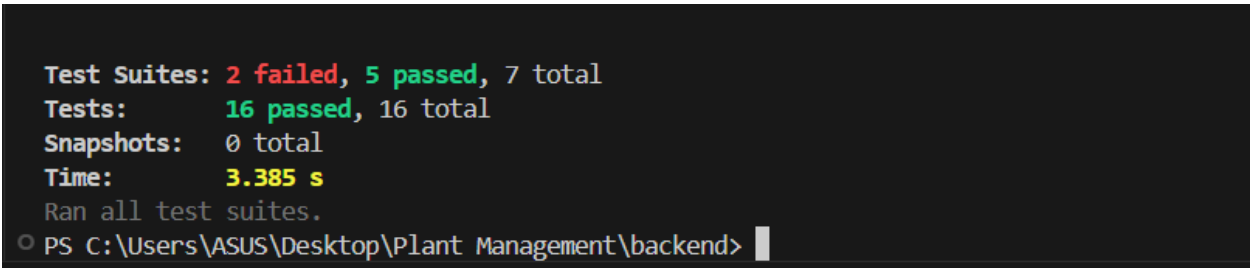
Unit Testing is performed to verify that individual components or modules of the application work as expected. Each function, API, or UI component is tested in isolation to ensure it returns the correct output for given inputs.

In this project, unit tests were written using Jest for the backend (controllers, routes, models) and Supertest for API testing. Tests were created for key features including CRUD operations on products, and authentication flows like login and registration. These tests help identify bugs early and improve code reliability. ([Getting Started · Jest, 2025](#))



```

▼ test
  ▼ mock
    JS authRoutes.test.js
    test.png
  JS productController.test.js
  JS productModel.test.js
  JS productRoutes.test.js
  JS security.test.js
  JS userController.test.js
  JS userModel.test.js
```



```

Test Suites: 2 failed, 5 passed, 7 total
Tests:      16 passed, 16 total
Snapshots:  0 total
Time:       3.385 s
Ran all test suites.
PS C:\Users\ASUS\Desktop\Plant Management\backend>
```

Test Case and Expected Results

Test cases were created to verify that each feature functioned properly. Expected results were used to confirm the system's behavior or identify any issues that needed attention.

Test Case ID	Test Scenario	Input	Expected Output	Actual Output	Status
TC001	User login with valid details	Email: test@example.com, Password: 123456	User successfully logged in, token saved	As expected	✓ Pass
TC002	User login with invalid password	Email: test@example.com, Password: wrongpass	Error: "Invalid credentials"	As expected	✓ Pass
TC003	User registration with valid data	Name, Email, Password, Phone, Gender	Registration successful	As expected	✓ Pass
TC004	User registration with mismatched passwords	Password: abc123, Confirm: xyz456	Error: "Passwords do not match"	As expected	✓ Pass
TC005	User logout (manual from dashboard)	Click logout, clear localStorage	Redirect to login, token removed	As expected	✓ Pass
TC006	Add product with valid input	Name: Orchid, Price: 200, Qty: 5, Image: orchid.jpg	Product successfully added	As expected	✓ Pass
TC007	Update product quantity	Edit Orchid product → Qty: 10	Product updated successfully	As expected	✓ Pass

TC008	Delete existing product	Product: Orchid, click delete	Product deleted and removed from table	As expected	✓ Pass
TC009	View product list	Navigate to Admin panel	Product list fetched via GET /api/products	As expected	✓ Pass
TC010	Add product with missing field	Missing: Price	Error or validation warning	As expected	✓ Pass

Future Work

1. Implement Payment Gateway Integration

Add secure payment options using Stripe or PayPal so users can make real-time purchases directly through the application.

2. Admin Dashboard Enhancements

Extend the admin panel to include features like user management, detailed analytics, product performance reports, and order tracking.

3. Cart and Checkout System

Develop a full shopping cart and checkout process, allowing users to view selected products, adjust quantities, and place orders.

4. User Profile & Order History

Allow users to update their profile information and view their previous orders, making the app more personalized and user-friendly.

5. Responsive and Progressive Web App (PWA)

Optimize the application for mobile devices and offline usage by converting it into a PWA for a smoother user experience.

6. Email Notifications and OTP Verification

Implement features like email verification during registration and order confirmation notifications to increase reliability and trust.

7. Add Product Reviews & Ratings

Let users provide feedback on products to build credibility and improve decision-making for other customers.



Figure 67: Future Work illustration

Conclusion

This project has successfully delivered a functional, responsive, and user-centric online plant shopping system. From user registration and secure login to browsing products, managing favorites, handling cart operations, and collecting delivery information, each feature was thoughtfully implemented to enhance the overall user experience. The integration between the frontend and backend was carefully managed to ensure smooth operation and data consistency throughout the system.

Beyond the technical achievements, this project provided valuable hands-on experience in full-stack development, team collaboration, and real-world problem-solving. It challenged us to think critically, communicate effectively, and apply our skills to build a system that is both practical and scalable.

While the current version meets the primary goals, there remains potential for future enhancements such as adding payment integration, real-time order tracking, and admin analytics. These improvements can further increase the system's functionality and user satisfaction.

In conclusion, this project stands as a strong representation of our technical capabilities, dedication, and growth throughout the development process. It not only fulfills the core objectives but also sets a solid foundation for future development and continuous learning.

References

The Ultimate Guide to Selling Plants Online. (2025, May 20). Shopify.

<https://www.shopify.com/blog/sell-plants-online>

Lakhani, V. (2025, May 26). *13 Problems faced by customers while shopping online and how to solve it.* Rock Technolabs. <https://rocktechnolabs.com/blog/common-problems-faced-by-customers-while-online-shopping/>

Agilemania. (n.d.). *Functional vs Nonfunctional Requirements: The 2025 Guide.* Functional Vs Nonfunctional Requirements: The 2025 Guide. <https://agilemania.com/functional-vs-nonfunctional-requirements>

Web design process explained in 8 easy steps. (n.d.). <https://www.softr.io/blog/web-design-process>

Gillis, A. S. (2024, October 22). *What is a 3-tier application architecture?* Search Software Quality. <https://www.techtarget.com/searchsoftwarequality/definition/3-tier-application>

What is a Tech Stack: Examples, Components, and Diagrams. (n.d.). Heap. <https://www.heap.io/topics/what-is-a-tech-stack>

Getting started · jest. (2025, June 10). <https://jestjs.io/docs/getting-started>

What is the difference between Front-End and Back-End development? (2024, July 17). <https://www.conceptatech.com/blog/difference-front-end-back-end-development>

Appendix

<https://www.figma.com/design/GuZoZYiHLr0opr1WTtH7X/plant-store?node-id=0-1&p=f&t=GVFPgdw1ksNGV6il-0>

<https://balsamiq.cloud/svjfnzk/pin3ppp/r2278>

<https://youtu.be/TWiSTddllpl>

<https://github.com/anishaii/Web-development/tree/main/frontend>

<https://github.com/anishaii/Web-development/tree/main/backend>